



Laboratory Guidelines

COURSE: _____

Learning Outcomes:

1. The art of experimentation
2. Experimental and analytical skills
3. Conceptual learning
4. Understanding the basis of Python and Software Defined Networks
5. Developing collaborative learning skills

Authors: Dr. Karina Gomez Chavez and Paul Zanna

Contents

Lab 1: Introduction to Python	4
1. Objectives	4
2. Theory.....	4
3. Methodology.....	5
4. Exercises.....	10
5. Questions	13
Lab 2: Programing with Python	14
1. Objectives	14
2. Theory.....	14
3. Methodology.....	16
4. Exercises.....	16
5. Questions	21
Lab 3: Python for Networking	22
1. Objectives	22
2. Theory.....	22
3. Methodology.....	24
4. Exercises.....	24
5. Questions	32
Lab 4: SDN Controllers and Mininet.....	33
1. Objectives	33
2. Theory.....	33
3. Methodology.....	35
4. Exercises.....	36
5. Questions	42
6. References	42
Lab 5: Zodiac OpenFlow Switch (Configure)	43
1. Objectives	43
2. Theory.....	43
3. Methodology.....	44
4. Exercises.....	46
5. Questions	49
6. References	49
Lab 6: OpenFlow Protocol.....	50
1. Objectives	50
2. Theory.....	50
3. Methodology.....	52
4. Exercises.....	54
5. Questions	59
6. References	59
Lab 7: Controller Rest API	61
1. Objectives	61
2. Theory.....	61
3. Methodology.....	62

4.	Exercises.....	63
5.	Questions	65
6.	References	65

Lab 1: Introduction to Python

1. Objectives

The objective of the lab 1 is to:

- Setup python environment for programing,
- Learn the basics of python,
- Create and run basic examples using python.

2. Theory

Definition of Python: The official definition of Python is:

Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.

Main Features of Python: The main features of Python are:

- ✓ **Simple:** Python is a simple and minimalistic language. This pseudo-code nature of Python is one of its greatest strengths.
- ✓ **Easy to Learn:** Python is extremely easy to get started with. Python has an extraordinarily simple syntax.
- ✓ **Free and Open Source:** Python is an example of FLOSS (Free/Libre and Open Source Software). In simple terms, you can freely distribute copies of this software, read its source code, make changes to it, use pieces of it in new free programs, and that you know you can do these things. FLOSS is based on the concept of a community which shares knowledge.
- ✓ **High-level Language:** When you write programs in Python, you never need to bother about the low-level details such as managing the memory used by your program, etc.
- ✓ **Portable:** Due to its open-source nature, Python has been ported (i.e. changed to make it work on) to many platforms. All your Python programs can work on any of these platforms without requiring any changes at all if you are careful enough to avoid any system-dependent features.
- ✓ **Multi-Platform:** Python can be used on Linux, Windows, FreeBSD, Macintosh, Solaris, OS/2, Amiga, AROS, AS/400, BeOS, OS/390, z/OS, Palm OS, QNX, VMS, Psion, Acorn RISC OS, VxWorks, PlayStation, Sharp Zaurus, Windows CE and even PocketPC.
- ✓ **Interpreted:** Python does not need compilation to binary. You just run the program directly from the source code. Internally, Python converts the source code into an

intermediate form called byte codes and then translates this into the native language of your computer and then runs it. All this, actually, makes using Python much easier since you don't have to worry about compiling the program, making sure that the proper libraries are linked and loaded, etc, etc. This also makes your Python programs much more portable, since you can just copy your Python program onto another computer and it just works!

- ✓ *Object Oriented*: Python supports procedure-oriented programming as well as object oriented programming. In procedure-oriented languages, the program is built around procedures or functions which are nothing but reusable pieces of programs. In object-oriented languages, the program is built around objects which combine data and functionality.
- ✓ *Extensible*: If you need a critical piece of code to run very fast or want to have some piece of algorithm not to be open, you can code that part of your programing C or C++ and then use them from your Python program.
- ✓ *Embeddable*: You can embed Python within your C/C++ programs to give 'scripting' capabilities for your program's users.
- ✓ *Extensive Libraries*: The Python Standard Library is huge indeed. It can help you do various things involving regular expressions, documentation generation, unit testing, threading, databases, web browsers, CGI, ftp, email, XML, XML-RPC, HTML, WAV files, cryptography, GUI (graphical user interfaces), Tk, and other system-dependent stuff. Remember, all this is always available wherever Python is installed.
- ✓ *Other Libraries*: Besides, the standard library, there are various other high-quality libraries such as:
 - wxPython [<http://www.wxpython.org>],
 - Twisted [<http://www.twistedmatrix.com/products/twisted>],
 - Python Imaging Library [<http://www.pythonware.com/products/pil/index.htm>]

3. Methodology

Section 3.1: Setup of Python Environment

- ✓ **STEP 1**: Open Eclipse and setup a correct access to Internet (This is required only in RMIT network). In order to set up Manual Proxy follow the instructions (see also figure 1):
 - a. Go to **Windows > Preferences > General > Network Connections**.
 - b. Change Active Provider to Manual.
 - c. Input proxy details, including username/password if required.
 - **Host**: proxy.rmit.edu.au
 - **Port**: 8080
 - **Username/password**: No required
 - d. Clear SOCKS proxy.
 - e. Restart Eclipse.

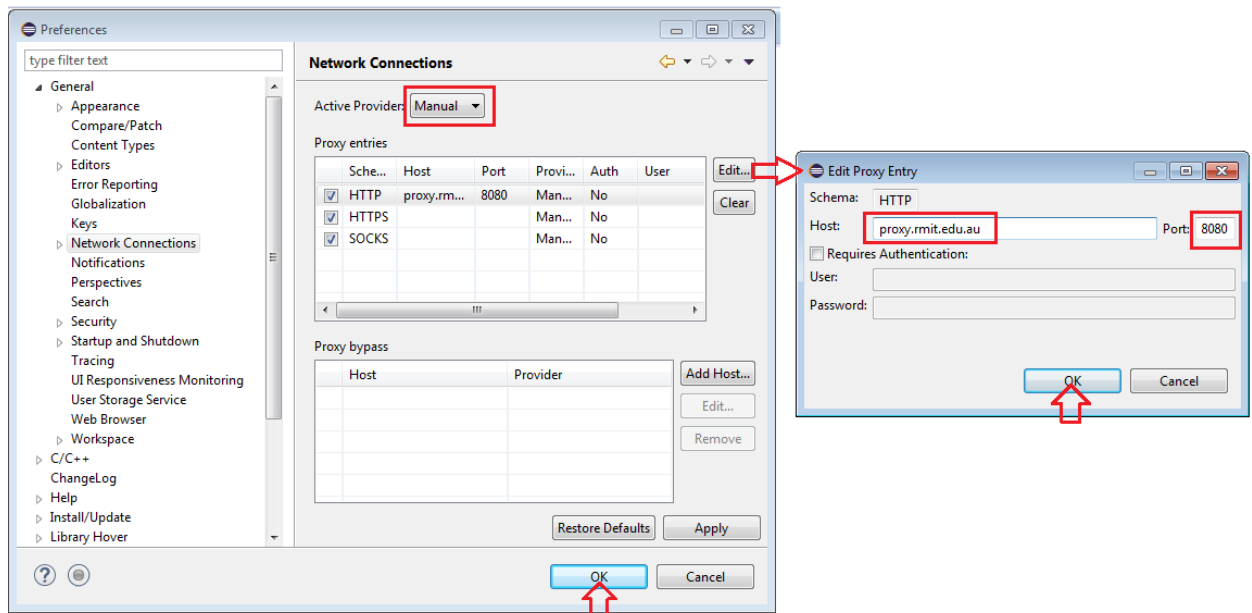


Figure 3-1. Eclipse setup for Internet.

- ✓ **STEP 2:** Installing python environment using Eclipse Graphical Interface¹.
 - a. To install PyDev and PyDev Extensions using the Eclipse Update Manager, you need to use the **Help > Install New Software...** menu (note that in older versions, this would be the 'Find and Install' menu) as shown in the following figure:

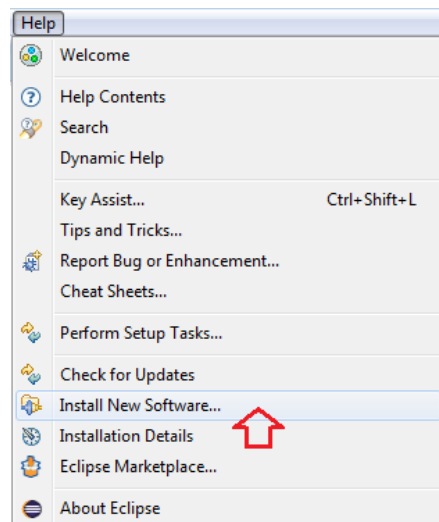


Figure 3-2. Step 2.

- b. In the next screen, add the update site(s) you want to work with (see the figure below). The available update sites are (see Figure 3):
 - <http://pydev.org/updates> (Last version for your personal PC)
 - <http://pydev.org/updates4.5> (version for RMIT Labs)

¹ **Note:** Instructions are targeted at Eclipse 3.5 onwards

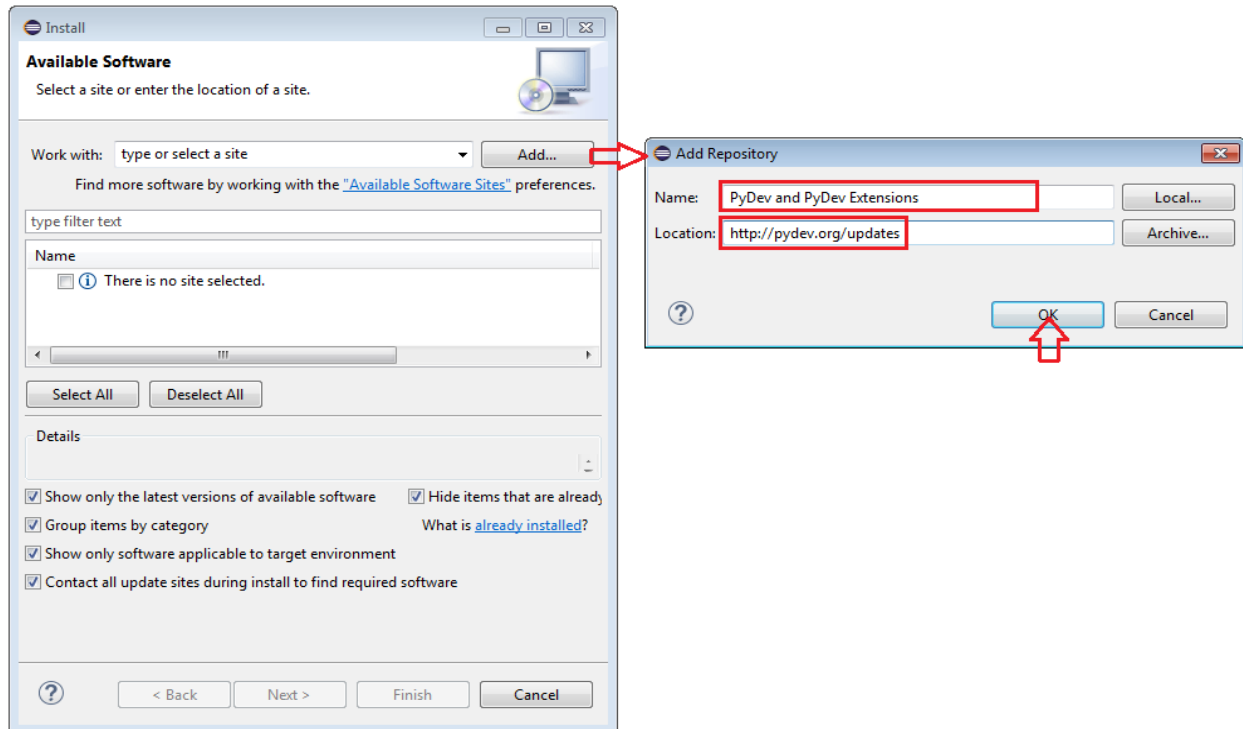


Figure 3-3. Set up Python on Eclipse.

- c. After entering the update sites, select the update site you entered or select "All available sites" and add a filter for PyDev, so that it shows the contents of all the update sites that have PyDev, then select what you want to install and click 'Next' (see Figure 4).

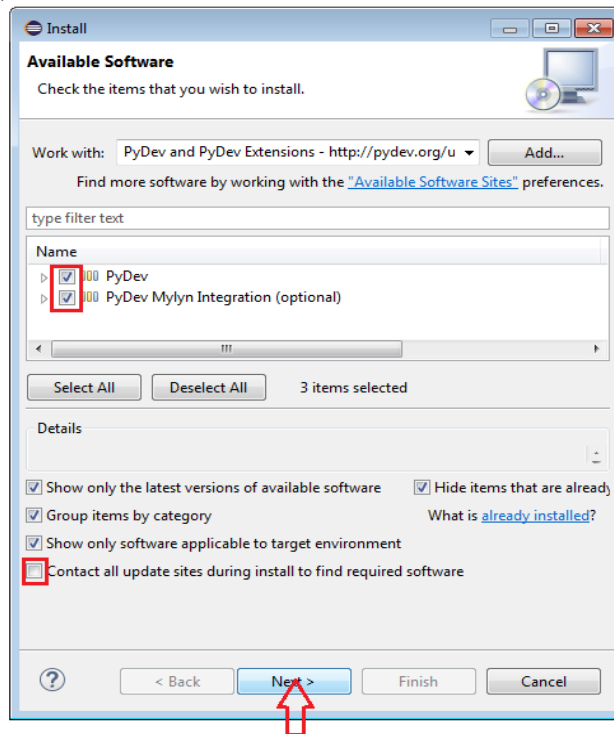


Figure 3-4. Set up Python on Eclipse.

- d. Then, UNCHECK the 'Contact all update sites during install to find required software' and press 'Next' again to confirm your selection (see Figure 5).

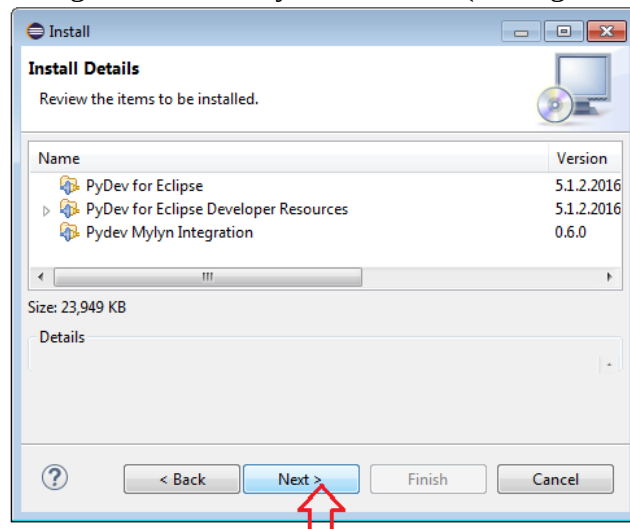


Figure 3-5. Set up Python on Eclipse.

- e. And finally, read the license agreement and if you accept, select the accept radio button and click 'Finish' (see Figure 6).

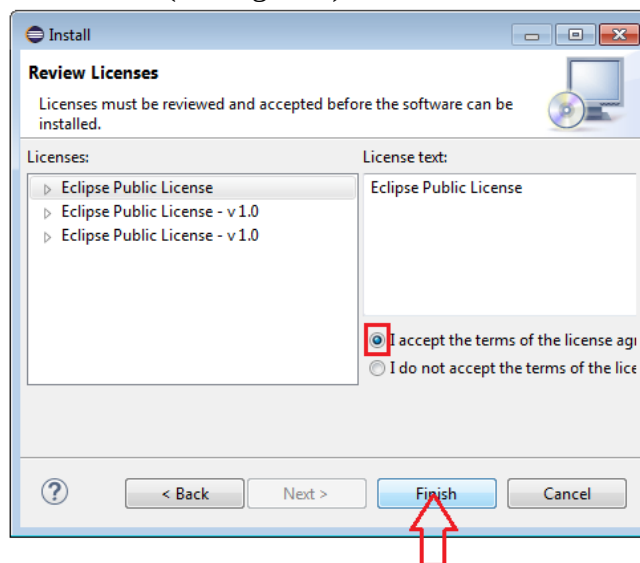


Figure 3-6. Set up Python on Eclipse.

- f. At that point, Eclipse should automatically download the plugin contents and present you to a dialog asking if you want to restart (to which you should say **yes**).
- ✓ **STEP 2: Checking the installation:** You can verify if it is correctly installed going to the menu '**window> preferences**' and checking if there is a **PyDev** item under that (see Figure 7). After that eclipse will display the graphical interface for python perspective, the main components are (see Figure 8):
- Project space is the section where all your python projects are visualized,
 - Project Editor is the section where python scripts can be edited,
 - Console allows the visualization of results father running a python script,

- Run bottom allows to run a python script,

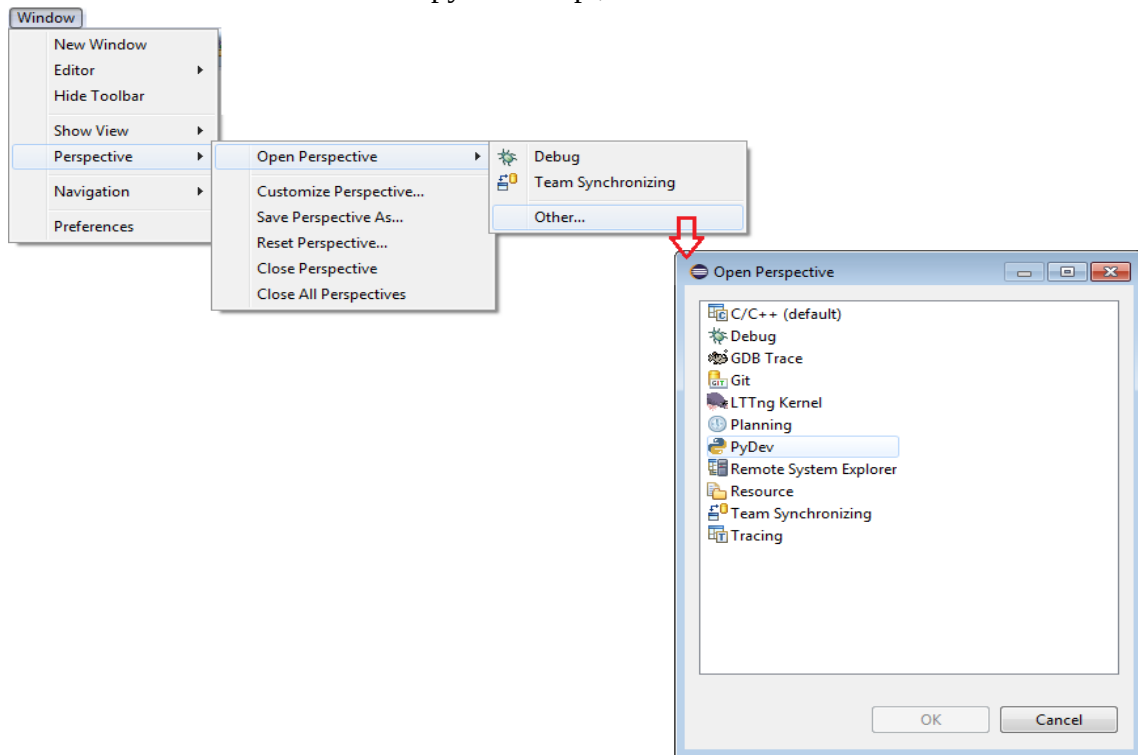


Figure 3-7. Python perspective in Eclipse.

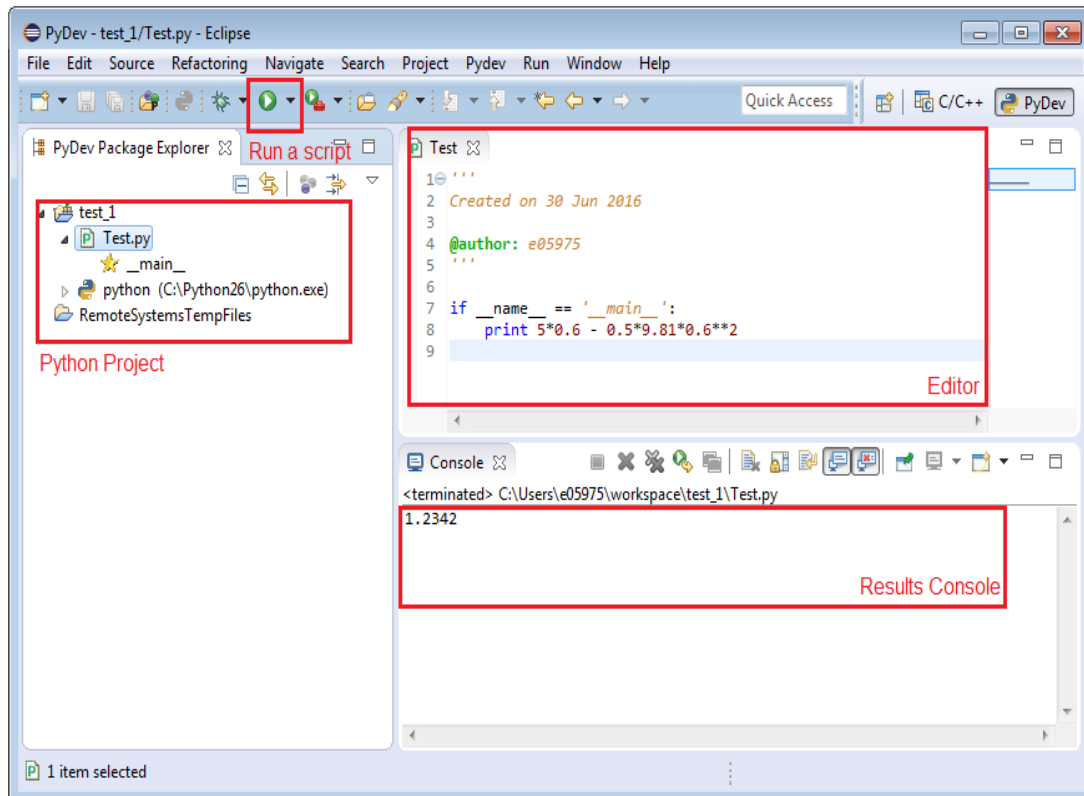
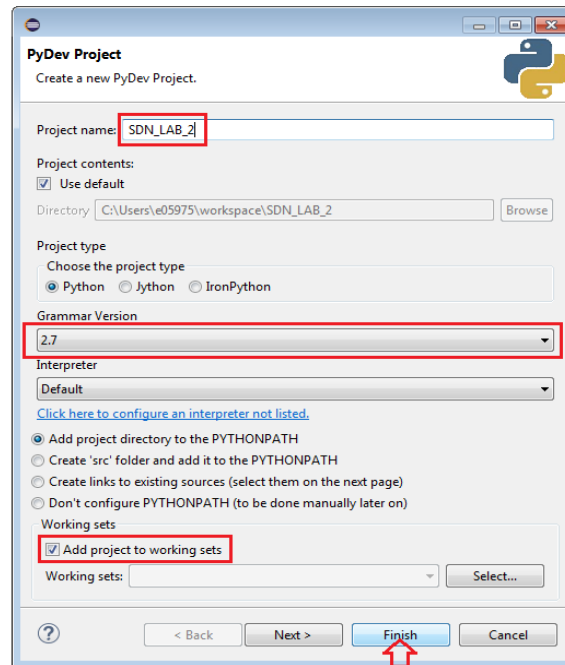


Figure 3-8. Python perspective environment.

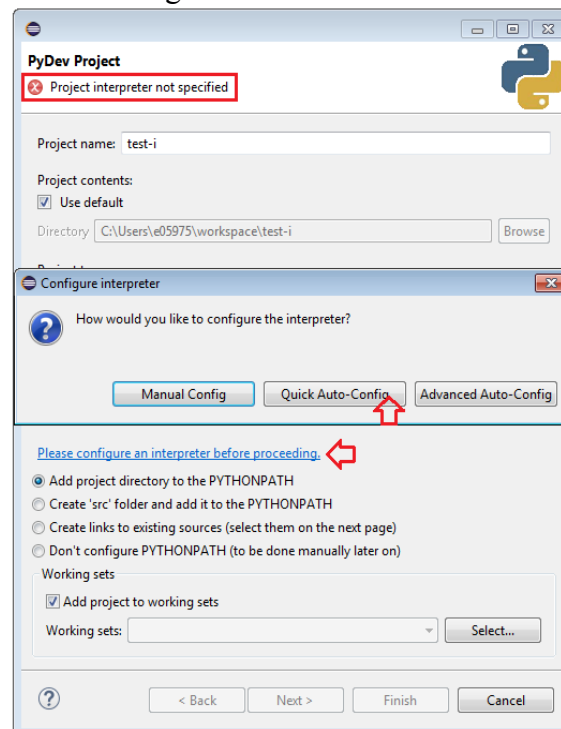
4. Exercises

Section 4.1: Basics of python and programming

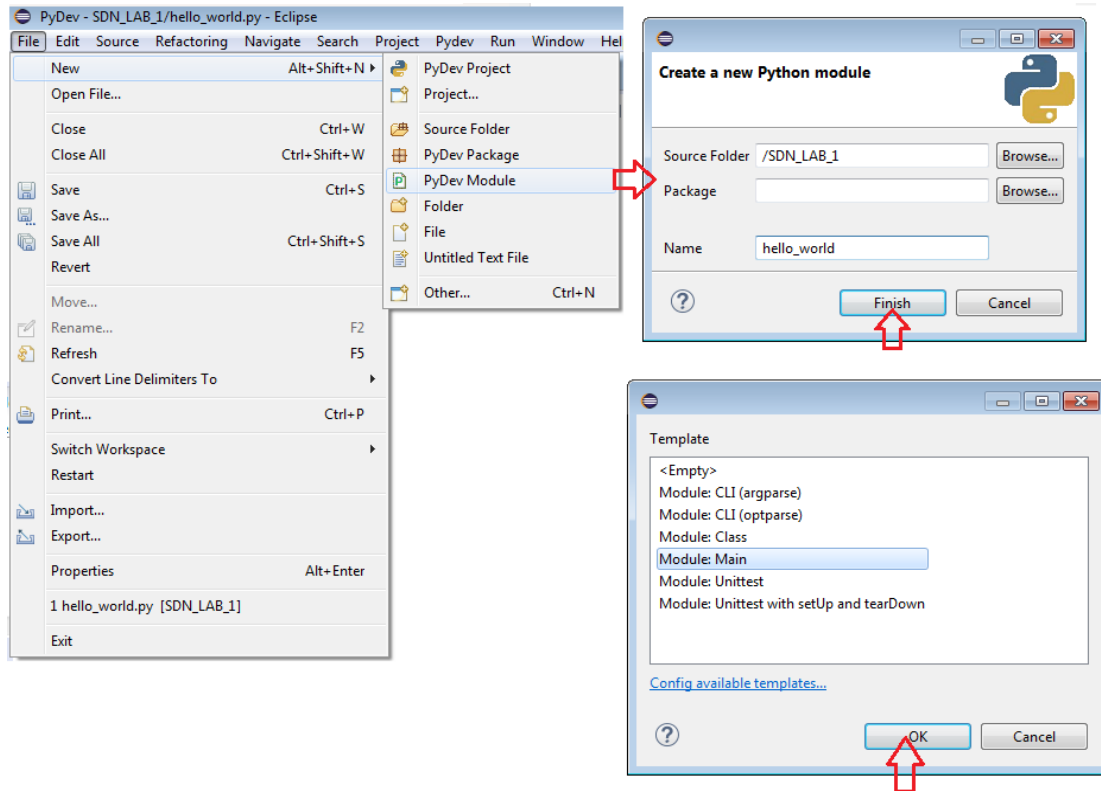
- **Exercise 4.1.1:** Create a python project, click in **File > New > PyDev Project**. Provide a name for the project (SDN_LAB_1 for the fits lab), then select the version of python to be used and select to add the project to working set as shown below:



Note: If the error project interpreter is not configure appear, click in “Please configure an interpreter > Quick Auto-Config” as shown below:



Create a python script, click in **File > New > PyDev Module**. Select the folder source name. Then, provide a name for the project (Hello_world), then select empty module or main module as shown below:



- **Exercise 4.1.2:** Write a Hello World program
Almost all books about programming languages start with a very simple program that prints the text Hello, World! to the screen. Make such a program in Python. (save as hello_world.py).
- **Exercise 4.1.3:** Compute 1+1
The first exercise concerns some very basic mathematics and programming: assign the result of 1+1 to a variable and print the value of that variable (save as 1plus1.py).
- **Exercise 4.1.4:** Type in program text
Type the following program in your editor and execute it. If your program does not work, check that you have copied the code correctly and debug it (save as formulas_shapes.py).

```
h = 5.0 # height
r = 1.5 # radius
if __name__ == '__main__':
    area_parallelogram = h*b
    print ('The area of the parallelogram is %.3f' % area_parallelogram)
    area_square = b**2
    print ('The area of the square is %g' % area_square)
    area_circle = pi*r**2
    print ('The area of the circle is %.3f' % area_circle)
    volume_cone = 1.0/3*pi*r**2*h

    print ('The volume of the cone is %.3f' % volume_cone)
```

Section 4.1: Create and run basic example.

- **Exercise 4.2.1:** Verify the use of the following operator. Execute the example code in python script and provide the output.

Operator	Name	Explanation	Examples
+	Plus	Adds two objects	3 + 5 'a' + 'b'
-	Minus	Gives the subtraction of one number from the other; if the first operand is absent it is assumed to be zero.	-5.2 50 - 24
*	Multiply	Gives the multiplication of the two numbers or returns the string repeated that many times.	2 * 3 'la' * 3
**	Power	Returns x to the power of y	3 ** 4
/	Divide	Divide x by y	13 / 3
//	Divide and floor	Divide x by y and round the answer down to the nearest whole number	13 // 3 -13 // 3
%	Modulo	Returns the remainder of the division	13 % 3 -25.5 % 2.25
<<	Left shift	Shifts the bits of the number to the left by the number of bits specified. (Each number is represented in memory by bits or binary digits i.e. 0 and 1)	2 << 2
>>	Right shift	Shifts the bits of the number to the right by the number of bits specified.	11 >> 1
&	Bit-wise AND	Bit-wise AND of the numbers	5 & 3
	Bit-wise OR	Bitwise OR of the numbers	5 3
^	Bit-wise XOR	Bitwise XOR of the numbers	5 ^ 3
~	Bit-wise invert	The bit-wise inversion of x is -(x+1)	~5
<	Less than	Returns whether x is less than y. All comparison operators return True or False.	5 < 3 3 < 5
>	Greater than	Returns whether x is greater than y	5 > 3
<=	Less than or equal to	Returns whether x is less than or equal to y	x = 3; y = 6; x <= y
>=	Greater than or equal to	Returns whether x is greater than or equal to y	x = 4; y = 3; x >= 3
==	Equal to	Compares if the objects are equal	x = 2; y = 2; x == y x = 'str'; y = 'stR'; x == y x = 'str'; y = 'str'; x == y
!=	Not equal to	Compares if the objects are not equal	x = 2; y = 3; x != y
not	Boolean NOT	If x is True, it returns False. If x is False, it returns True.	x = True; not x
and	Boolean AND	x and y returns False if x is False, else it returns evaluation of y	x = False; y = True; x and y
or	Boolean OR	If x is True, it returns True, else it returns evaluation of y	x = True; y = False; x or y

- **Exercise 4.2.2:** The if statement:

Create a program for taking a number from the user and check if it is the number that you have saved in the code (TIP: use input command). Save the file as if.py

- The if statement is used to check a condition: if the condition is true, we run a block of statements (called the if-block), else we process another block of statements (called the else-block). The else clause is optional.

- **Exercise 4.2.3:** The while Statement

Create a program for taking a number from the user and check if it is the number that you have saved in the code. The program run until the user will guess the number. Save the file as while.py

- The while statement allows you to repeatedly execute a block of statements as long as a condition is true. A while statement is an example of what is called a looping statement. A while statement can have an optional else clause.

- **Exercise 4.2.4:** The for Statement

Create a program for printing a sequence of numbers. Save the file as for.py

The *for..in* statement is another looping statement which iterates over a sequence of objects i.e. go through each item in a sequence.

5. Questions

- **Question 5.1:** Explain what is eclipse? And why we use it for programing on python?
- **Question 5.2:** Explain three main characteristics of python that you test in the lab?
- **Question 5.3:** Which is the difference between empty module and main module when creating a python script?
- **Question 5.4:** Find error(s) in a program
Suppose somebody has written a simple one-line program for computing sin(1):

```
x=1; print 'sin(%g)=%g' % (x, sin(x))
```

Create this program and try to run it. What is the problem? Which is the correct code?

- **Question 5.5:** Create a python program that combines at least 4 operators and one statement (if, while or for)

Lab 2: Programing with Python

1. Objectives

The objective of the lab 2 is to:

- Understand how python function works
- Understand the use of global and local variables
- Understand how python modules works
- Learning the basis of networking programing with python

2. Theory

Python functions: Functions are reusable pieces of programs. They allow you to give a name to a block of statements, allowing you to run that block using the specified name anywhere in the program and any number of times. This is known as calling the function.

Local Variables: Variables declared inside a function definition are not related in any way to other variables with the same names used outside the function (variable names are local to the function). This is called the scope of the variable. All variables have the scope of the block they are declared in starting from the point of definition of the name.

The global statement: Variables defined at the top level of the program are intended global. Global variables are intended to be used in any functions or classes). Global statement allows defining global variables inside functions as well.

Modules: Modules allow reusing a number of functions in other programs.

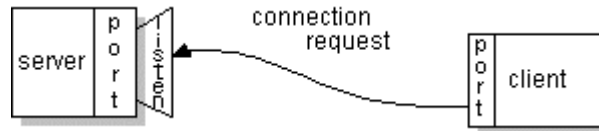
Networking background for sockets

What is a socket and how use it? A socket is one endpoint of a two-way communication link between two programs running on the network or PC. A socket is bound to a port number so that the TCP layer can identify the application that data is destined to be sent to.

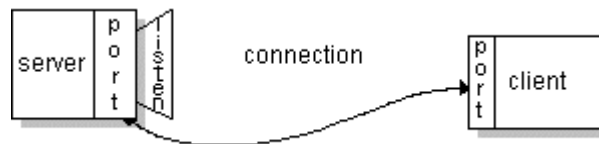
Endpoint: An endpoint is a combination of an IP address and a port number.

Server and Client: Normally, a server runs on a specific computer and has a socket that is bound to a specific port number.

- **On the server-side:** The server just waits, listening to the socket for a client to make a connection request.
- **On the client-side:** The client knows the hostname of the machine on which the server is running and the port number on which the server is listening. To make a connection request, the client tries to rendezvous with the server on the server's machine and port. The client also needs to identify itself to the server so it binds to a local port number that it will use during this connection. This is usually assigned by the system.



If everything goes well, the server accepts the connection. Upon acceptance, the server gets a new socket bound to the same local port and also has its remote endpoint set to the address and port of the client. It needs a new socket so that it can continue to listen to the original socket for connection requests while tending to the needs of the connected client.



On the client side, if the connection is accepted, a socket is successfully created and the client can use the socket to communicate with the server. The client and server can now communicate by writing to or reading from their sockets.

- **TCP:** TCP stands for transmission control protocol. It is implemented in the transport layer of the IP/TCP model and is used to establish reliable connections. TCP is one of the protocols that encapsulate data into packets. It then transfers these to the remote end of the connection using the methods available on the lower layers. On the other end, it can check for errors, request certain pieces to be resent, and reassemble the information into one logical piece to send to the application layer.

The protocol builds up a connection prior to data transfer using a system called a three-way handshake. This is a way for the two ends of the communication to acknowledge the request and agree upon a method of ensuring data reliability. After the data has been sent, the connection is torn down using a similar four-way handshake.

TCP is the protocol of choice for many of the most popular uses for the internet, including WWW, FTP, SSH, and email. It is safe to say that the internet we know today would not be here without TCP.

- **UDP:** UDP stands for user datagram protocol. It is a popular companion protocol to TCP and is also implemented in the transport layer.

The fundamental difference between UDP and TCP is that UDP offers unreliable data transfer. It does not verify that data has been received on the other end of the connection. This might sound like a bad thing, and for many purposes, it is. However, it is also extremely important for some functions.

Because it is not required to wait for confirmation that the data was received and forced to resend data, UDP is much faster than TCP. It does not establish a connection with the remote host, it simply fires off the data to that host and doesn't care if it is accepted or not. Because it is a simple transaction, it is useful for simple communications like querying for network resources. It also doesn't maintain a state, which makes it great for transmitting data from one

machine to many real-time clients. This makes it ideal for VOIP, games, and other applications that cannot afford delays.

3. Methodology

Defining functions: Functions are defined using the **def** keyword. After this keyword comes an identifier name for the function, followed by a pair of parentheses which may enclose some names of variables, and by the final colon that ends the line.

```
def XX_YY(variable1, variable2):  
    # block belonging to the function  
    # End of function
```

Defining local and global variables: Local and global variables can be defined using:

```
x = 50 #Local  
global x
```

Defining modules: There are various methods of writing modules, but the simplest way is to create a file with a .py extension that contains functions and variables.

```
def xx_yy():  
    aa
```

Using modules: A module can be imported by another program to make use of its functionality. This is how we can use the Python standard library as well.

```
import xx_yy
```

4. Exercises

Section 4.1: Python function variables and modules.

- **Exercise 4.1.1:** Create a python project using with SDN_LAB
- **Exercise 4.1.2:** Python function (save as function.py)

Create python scrip using the syntax provided below.

```
def say_hello():  
    # block belonging to the function  
    print('hello world')  
    # End of function  
  
if __name__ == '__main__':  
    say_hello() # call the function
```

Which is the output of this function? Does the function need any parameter?

- **Exercise 4.1.3:** Python function (save as function_2.py)

Create python scrip using the syntax provided below.

```
def print_max(a, b):  
    if a > b:  
        print(a, 'is maximum')  
    elif a == b:  
        print(a, 'is equal to', b)  
    else:
```



```

        print(b, 'is maximum')

if __name__ == '__main__':
    pass
    print_max(3, 4)
    # directly pass literal values
    x = 5
    y = 7
    # pass variables as arguments
    print_max(x, y)

```

Which is the output of this function? Does the function need any parameter?

- **Exercise 4.1.4:** Local variable (save as function_local.py)

Create python scrip using the syntax provided below.

```

x = 50

def func(x):
    print('x is', x)
    x = 2
    print('Changed Local x to', x)

if __name__ == '__main__':
    func(x)
    print('x is still', x)

```

Which is the final value of variable x? Why variable x does not change to 2?

- **Exercise 4.1.5:** Global variable (save as function_global.py)

Create python scrip using the syntax provided below.

```

x = 50

def func():
    global x
    print('x is', x)
    x = 2
    print('Changed global x to', x)

if __name__ == '__main__':
    func()
    print('Value of x is', x)

```

Which is the final value of variable x? Why variable x change this time?

- **Exercise 4.1.6:** Python modules

Create python scrip using the syntax provided below (save as mymodule.py).

```

def say_hi():
    print('Hi, this is mymodule speaking.')

__version__ = '0.1'

```

Create python scrip using the syntax provided below (save as module_demo.py).

```

import mymodule

if __name__ == '__main__':
    mymodule.say_hi()
    print('Version', mymodule.__version__)

```

Run the script, which is the role of **import**?

Create python scrip using the syntax provided below (save as module_demo2.py).

```

from mymodule import say_hi, __version__

if __name__ == '__main__':
    say_hi()
    print('Version', __version__)

```

Run the script, which is the role of **from, import**?

Section 4.2: Sockets, IPv4, and Simple Client/Server Programming

- **Exercise 4.2.1:** Printing your machine's name and IPv4 address

Create python scrip using the syntax provided below (save as local_machine_info.py):

```

import socket

def print_machine_info():
    host_name = socket.gethostname()
    ip_address = socket.gethostbyname(host_name)
    print ("    Host name: %s" % host_name)
    print ("    IP address: %s" % ip_address)
if __name__ == '__main__':
    print_machine_info()

```

Run the script, which module the program uses? Provide two additional functions of socket?

- **Exercise 4.2.2:** Retrieving a remote machine's IP address

Create python scrip using the syntax provided below (save as remote_machine_info.py):

```

import socket

def get_remote_machine_info():
    remote_host = 'www.python.org'
    try:
        print ("    Remote host name: %s" % remote_host)
        print ("    IP address: %s" % socket.gethostbyname(remote_host))
    except socket.error as err_msg:
        print ("Error accesing %s: error number and detail %s"
              %(remote_host, err_msg))

if __name__ == '__main__':
    get_remote_machine_info()

```

Run the script, which is the output? Modify the code for getting the RMIT website info.

- **Exercise 4.2.3:** Converting an IPv4 address to different formats

Create python scrip using the syntax below (save as ip4_address_conversion.py):

```

import socket
from binascii import hexlify

def convert_ip4_address():
    for ip_addr in ['127.0.0.1', '192.168.0.1']:
        packed_ip_addr = socket.inet_aton(ip_addr)
        unpacked_ip_addr = socket.inet_ntoa(packed_ip_addr)
        print ("    IP Address: %s => Packed: %s, Unpacked: %s"
              %(ip_addr, hexlify(packed_ip_addr), unpacked_ip_addr))

if __name__ == '__main__':
    convert_ip4_address()

```

Run the script, which is the output? How **binascii** works?

- **Exercise 4.2.4:** Finding a service name, given the port and protocol

Create python scrip using the syntax below (save as finding_service_name.py):

```
import socket
def find_service_name():
    protocolname = 'tcp'
    for port in [80, 25]:
        print ("Port: %s => service name: %s" %(port,
socket.getservbyport(port, protocolname)))
        print ("Port: %s => service name: %s" %(53,
socket.getservbyport(53, 'udp')))

if __name__ == '__main__':
    find_service_name()
```

Run the script, which is the output? Modify the code for getting complete the table:

Port	Protocol Name
21	
22	
110	

- **Exercise 4.2.5:** Setting and getting the default socket timeout

Create python scrip using the syntax below (save as socket_timeout.py):

```
import socket

def test_socket_timeout():
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    print ("Default socket timeout: %s" %s.gettimeout())
    s.settimeout(100)
    print ("Current socket timeout: %s" %s.gettimeout())

if __name__ == '__main__':
    test_socket_timeout()
```

Run the script, which is the role of socket timeout in real applications?

- **Exercise 4.2.6:** Writing a simple echo client/server application (**Tip:** Use port 9900)

Create python scrip using the syntax below (save as echo_server.py):

```
import socket
import sys
import argparse
import codecs

from codecs import encode, decode
host = 'localhost'
data_payload = 4096
backlog = 5

def echo_server(port):
    """ A simple echo server """
    # Create a TCP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # Enable reuse address/port
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    # Bind the socket to the port
```

```

server_address = (host, port)
print ("Starting up echo server on %s port %s" %server_address)
sock.bind(server_address)
# Listen to clients, backlog argument specifies the max no. of queued
connections
sock.listen(backlog)

while True:
    print ("Waiting to receive message from client")
    client, address = sock.accept()
    data = client.recv(data_payload)
    if data:
        print ("Data: %s" %data)
        client.send(data)
        print ("sent %s bytes back to %s" % (data, address))
    # end connection
    client.close()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port', action="store", dest="port", type=int,
required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_server(port)

```

Create python scrip using the syntax below (save as echo_client.py):

```

#!/usr/bin/env python

import socket
import sys
import argparse
import codecs

from codecs import encode, decode

host = 'localhost'

def echo_client(port):
    """ A simple echo client """
    # Create a TCP/IP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # Connect the socket to the server
    server_address = (host, port)
    print ("Connecting to %s port %s" % server_address)
    sock.connect(server_address)
    # Send data
    try:
        # Send data
        message = "Test message: SDN course examples"
        print ("Sending %s" % message)
        sock.sendall(message.encode('utf_8'))
        # Look for the response

```

```

        amount_received = 0
        amount_expected = len(message)
        while amount_received < amount_expected:
            data = sock.recv(16)
            amount_received += len(data)
            print ("Received: %s" % data)
        except socket.errno as e:
            print ("Socket error: %s" %str(e))
        except Exception as e:
            print ("Other exception: %s" %str(e))
        finally:
            print ("Closing connection to the server")
            sock.close()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port', action="store", dest="port", type=int,
        required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_client(port)

```

- Run the script, which is the output?
- What you need to do for running the program?
- Which program you need to run first client of server?
- Explain how the program works?
- What you need to do for communicating with another server in the classroom?

5. Questions

- **Question 5.1:** Explain in your own words which are the difference between functions and modules?
- **Question 5.2:** Explain in your own words when to use local and global variables?
- **Question 5.3:** Which is the role of sockets in computing networking? Are the sockets defined random or there is a rule?
- **Question 5.4:** Why is relevant to have the IPv4 address of remote server? Explain what is [Domain Name System](#) (DNS)?
- **Question 5.5:** Create a program that allows exchange messages increased by the user between client and server.

Lab 3: Python for Networking

1. Objectives

The objective of the lab 3 is to:

- Install python and use third-party libraries
- Interact with network interfaces using python
- Getting information from internet using Python

2. Theory

Third-party libraries: Although the Python's standard library provides a great set of awesome functionalities, there will be times that you will eventually run into the need of making use of third party libraries. Can you imagine building a webserver from scratch? Or making a port to a database driver? Or, maybe, comming up with an image manipulation tool?. Third party libraries are welcome in a way that they prevent you from reinventing the something that exit. They save you time to focus on finishing and delivering your application.

The following is a list of those third-party libraries with their download URLs used in networking:

- ntplib: <https://pypi.python.org/pypi/ntplib/>
- diesel: <https://pypi.python.org/pypi/diesel/>
- nmap: <https://pypi.python.org/pypi/python-nmap>
- scapy: <https://pypi.python.org/pypi/scapy>
- netifaces: <https://pypi.python.org/pypi/netifaces/>
- netaddr: <https://pypi.python.org/pypi/netaddr>
- pyopenssl: <https://pypi.python.org/pypi/pyOpenSSL>
- pygeocoder: <https://pypi.python.org/pypi/pygocoder>
- pyyaml: <https://pypi.python.org/pypi/PyYAML>
- requests: <https://pypi.python.org/pypi/requests>
- feedparser: <https://pypi.python.org/pypi/feedparser>
- paramiko: <https://pypi.python.org/pypi/paramiko/>
- fabric: <https://pypi.python.org/pypi/Fabric>
- supervisor: <https://pypi.python.org/pypi/supervisor>
- xmlrpclib: <https://pypi.python.org/pypi/xmlrpclib>
- SOAPpy: <https://pypi.python.org/pypi/SOAPpy>
- bottlenose: <https://pypi.python.org/pypi/bottlenose>
- construct: <https://pypi.python.org/pypi/construct/>
- pyserial: <https://pypi.python.org/pypi/pyserial>

Networking Glossary: Before we begin discussing networking with any depth, we must define some common terms that you will see throughout this guide, and in other guides and documentation regarding networking.

- **Connection:** In networking, a connection refers to pieces of related information that are transferred through a network. This generally infers that a connection is built before the data transfer (by following the procedures laid out in a protocol) and then is deconstructed at the end of the data transfer.

- **Packet:** A packet is, generally speaking, the most basic unit that is transferred over a network. When communicating over a network, packets are the envelopes that carry your data (in pieces) from one end point to the other. Packets have a header portion that contains information about the packet including the source and destination, timestamps, network hops, etc. The main portion of a packet contains the actual data being transferred. It is sometimes called the body or the payload.
- **Network Interface:** A network interface can refer to any kind of software interface to networking hardware. For instance, if you have two network cards in your computer, you can control and configure each network interface associated with them individually. A network interface may be associated with a physical device, or it may be a representation of a virtual interface. The "loopback" device, which is a virtual interface to the local machine, is an example of this.
- **LAN:** LAN stands for "local area network". It refers to a network or a portion of a network that is not publicly accessible to the greater internet. A home or office network is an example of a LAN.
- **WAN:** WAN stands for "wide area network". It means a network that is much more extensive than a LAN. While WAN is the relevant term to use to describe large, dispersed networks in general, it is usually meant to mean the internet, as a whole. If an interface is said to be connected to the WAN, it is generally assumed that it is reachable through the internet.
- **Protocol:** A protocol is a set of rules and standards that basically define a language that devices can use to communicate. There are a great number of protocols in use extensively in networking, and they are often implemented in different layers. Some low level protocols are TCP, UDP, IP, and ICMP. Some familiar examples of application layer protocols, built on these lower protocols, are HTTP (for accessing web content), SSH, TLS/SSL, and FTP. **Port:** A port is an address on a single machine that can be tied to a specific piece of software. It is not a physical interface or location, but it allows your server to be able to communicate using more than one application.
- **Firewall:** A firewall is a program that decides whether traffic coming into a server or going out should be allowed. A firewall usually works by creating rules for which type of traffic is acceptable on which ports. Generally, firewalls block ports that are not used by a specific application on a server.
- **NAT:** NAT stands for network address translation. It is a way to translate requests that are incoming into a routing server to the relevant devices or servers that it knows about in the LAN. This is usually implemented in physical LANs as a way to route requests through one IP address to the necessary backend servers.

- **VPN:** VPN stands for virtual private network. It is a means of connecting separate LANs through the internet, while maintaining privacy. This is used as a means of connecting remote systems as if they were on a local network, often for security reasons.
- **Interfaces:** Interfaces are networking communication points for your computer. Each interface is associated with a physical or virtual networking device. Typically, your server will have one configurable network interface for each Ethernet or wireless internet card you have.
In addition, it will define a virtual network interface called the "loopback" or localhost interface. This is used as an interface to connect applications and processes on a single computer to other applications and processes. You can see this referenced as the "lo" interface in many tools. Many times, administrators configure one interface to service traffic to the internet and another interface for a LAN or private network.
- **Protocols:** Networking works by piggybacking a number of different protocols on top of each other. In this way, one piece of data can be transmitted using multiple protocols encapsulated within one another. We will talk about some of the more common protocols that you may come across and attempt to explain the difference, as well as give context as to what part of the process they are involved with. We will start with protocols implemented on the lower networking layers and work our way up to protocols with higher abstraction.

3. Methodology

Installing Python Third-party includes:

- ✓ Python Third-party includes a setup.py file, it is usually distributed as a tarball (.tar.gz or .tar.bz2 file). The instructions for installing these generally look like:
 - Download the file from website.
 - Extract the tarball.
 - Change into the new directory that has been newly extracted.
 - Run `sudo python setup.py build`
 - Run `sudo python setup.py install`

4. Exercises

When importing a module if there is an error it means that the module needs to be installed.

- **Exercise 4.1:** Enumerating interfaces on your machine
Create python scrip using the syntax below (save as `list_network_interfaces.py`):

```
#!/usr/bin/env python
import sys
import socket
import fcntl
import struct
import array

SIOCGIFCONF = 0x8912 #from C library sockios.h
STUCT_SIZE_32 = 32
STUCT_SIZE_64 = 40
```



```

PLATFORM_32_MAX_NUMBER = 2**32
DEFAULT_INTERFACES = 8

def list_interfaces():
    interfaces = []
    max_interfaces = DEFAULT_INTERFACES
    is_64bits = sys.maxsize > PLATFORM_32_MAX_NUMBER
    struct_size = STUCT_SIZE_64 if is_64bits else STUCT_SIZE_32
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    while True:
        bytes = max_interfaces * struct_size
        interface_names = array.array('B', '\0' * bytes)
        sock_info = fcntl.ioctl(sock.fileno(), SIOCGIFCONF, struct.pack('iL',
bytes, interface_names.buffer_info()[0]))
        outbytes = struct.unpack('iL', sock_info)[0]
        if outbytes == bytes:
            max_interfaces *= 2
        else:
            break
    namestr = interface_names.tostring()
    for i in range(0, outbytes, struct_size):
        interfaces.append((namestr[i:i+16].split('\0', 1)[0]))
    return interfaces

if __name__ == '__main__':
    interfaces = list_interfaces()
    print ("This machine has %s network interfaces: %s."
%(len(interfaces), interfaces))

```

Run the script, which is the output? Verify the output using the command line **ifconfig**, which is the output?

- **Exercise 4.2:** Finding the IP address for a specific interface on your machine
Create python scrip using the syntax below (save as get_interface_ip_address.py):

```

import argparse
import sys
import socket
import fcntl
import struct
import array

def get_ip_address(ifname):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    return socket.inet_ntoa(fcntl.ioctl(s.fileno(), 0x8915,
struct.pack('256s', ifname[:15]))[20:24])

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Python networking utils')
    parser.add_argument('--ifname', action="store", dest="ifname",
required=True)
    given_args = parser.parse_args()
    ifname = given_args.ifname
    print ("Interface [%s] --> IP: %s" %(ifname, get_ip_address(ifname)))

```

Run the script, which is the output? Which variable you need to provide? What is the purpose of parse module?

- **Exercise 4.3:** Finding whether an interface is up on your machine

Create python scrip using the syntax below (save as find_network_interface_status.py):

```
import argparse
import socket
import struct
import fcntl
import nmap

SAMPLE_PORTS = '21-23'

def get_interface_status(ifname):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    ip_address = socket.inet_ntoa(fcntl.ioctl(sock.fileno(), 0x8915,
    struct.pack('256s', ifname[:15]))[20:24])
    nm = nmap.PortScanner()
    nm.scan(ip_address, SAMPLE_PORTS)
    return nm[ip_address].state()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Python networking utils')
    parser.add_argument('--ifname', action="store", dest="ifname",
    required=True)
    given_args = parser.parse_args()
    ifname = given_args.ifname
    print ("Interface [%s] is: %s" %(ifname, get_interface_status(ifname)))
```

Run the script providing the interface to be tested, which is the output? Write a script that provides the interfaces, IP and status (save as exercise43_solution.py)?

- **Exercise 4.4:** Detecting inactive machines on your network

Create python scrip using the syntax below (save as detect_inactive_machines.py):

```
import argparse
import time
import sched
from scapy.layers.inet import sr, srp, IP, UDP, ICMP, TCP, ARP, Ether
#from scapy.all import sr, srp, IP, UDP, ICMP, TCP, ARP, Ether

RUN_FREQUENCY = 10

scheduler = sched.scheduler(time.time, time.sleep)

def detect_inactive_hosts(scan_hosts):
    """
    Scans the network to find scan_hosts are Live or dead
    scan_hosts can be like 10.0.2.2-4 to cover range.
    See Scapy docs for specifying targets.
    """
    global scheduler
    scheduler.enter(RUN_FREQUENCY, 1, detect_inactive_hosts, (scan_hosts, ))
    inactive_hosts = []
    try:
```

```

ans, unans = sr(IP(dst=scan_hosts)/ICMP(), retry=0, timeout=1)
ans.summary(lambda(s,r) : r.sprintf("%IP.src% is alive"))
for inactive in unans:
    print ("%s is inactive" %inactive.dst)
    inactive_hosts.append(inactive.dst)

print ("Total %d hosts are inactive" %(len(inactive_hosts)))

except KeyboardInterrupt:
    exit(0)

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description='Python networking utils')
    parser.add_argument('--scan-hosts', action="store", dest="scan_hosts",
        required=True)
    given_args = parser.parse_args()
    scan_hosts = given_args.scan_hosts
    scheduler.enter(1, 1, detect_inactive_hosts, (scan_hosts, ))
    scheduler.run()

```

Add a range of IP address and run the script, which is the output? Ask your tutor for help.

Exercise 4.5: Ping hosts on the network with ICMP

Create python scrip using the syntax below (save as ping_remote_host.py):

```

#!/usr/bin/env python
import os
import argparse
import socket
import struct
import select
import time

ICMP_ECHO_REQUEST = 8 # Platform specific
DEFAULT_TIMEOUT = 2
DEFAULT_COUNT = 4

class Pinger(object):
    """ Pings to a host -- the Pythonic way """

    def __init__(self, target_host, count=DEFAULT_COUNT,
        timeout=DEFAULT_TIMEOUT):
        self.target_host = target_host
        self.count = count
        self.timeout = timeout

    def do_checksum(self, source_string):
        """ Verify the packet integrity """
        sum = 0
        max_count = (len(source_string)/2)*2

```

```

        count = 0
        while count < max_count:
            val = ord(source_string[count + 1])*256 +
ord(source_string[count])
            sum = sum + val
            sum = sum & 0xffffffff
            count = count + 2

        if max_count < len(source_string):
            sum = sum + ord(source_string[len(source_string) - 1])
            sum = sum & 0xffffffff

        sum = (sum >> 16) + (sum & 0xffff)
        sum = sum + (sum >> 16)
        answer = ~sum
        answer = answer & 0xffff
        answer = answer >> 8 | (answer << 8 & 0xff00)
        return answer

def receive_pong(self, sock, ID, timeout):
    """
    Receive ping from the socket.
    """
    time_remaining = timeout
    while True:
        start_time = time.time()
        readable = select.select([sock], [], [], time_remaining)
        time_spent = (time.time() - start_time)
        if readable[0] == []: # Timeout
            return

        time_received = time.time()
        recv_packet, addr = sock.recvfrom(1024)
        icmp_header = recv_packet[20:28]
        type, code, checksum, packet_ID, sequence = struct.unpack(
            "bbHHh", icmp_header
        )
        if packet_ID == ID:
            bytes_In_double = struct.calcsize("d")
            time_sent = struct.unpack("d", recv_packet[28:28 +
bytes_In_double])[0]
            return time_received - time_sent

        time_remaining = time_remaining - time_spent
        if time_remaining <= 0:
            return

def send_ping(self, sock, ID):
    """
    Send ping to the target host
    """
    target_addr = socket.gethostbyname(self.target_host)

    my_checksum = 0

```

```

# Create a dummy heder with a 0 checksum.
header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, my_checksum,
ID, 1)
bytes_In_double = struct.calcsize("d")
data = (192 - bytes_In_double) * "Q"
data = struct.pack("d", time.time()) + data

# Get the checksum on the data and the dummy header.
my_checksum = self.do_checksum(header + data)
header = struct.pack(
    "bbHHh", ICMP_ECHO_REQUEST, 0, socket.htons(my_checksum), ID, 1
)
packet = header + data
sock.sendto(packet, (target_addr, 1))

def ping_once(self):
    """
    Returns the delay (in seconds) or none on timeout.
    """
    icmp = socket.getprotobyname("icmp")
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, icmp)
    except socket.error, (errno, msg):
        if errno == 1:
            # Not superuser, so operation not permitted
            msg += "ICMP messages can only be sent from root user
processes"
            raise socket.error(msg)
    except Exception, e:
        print "Exception: %s" %(e)

    my_ID = os.getpid() & 0xFFFF

    self.send_ping(sock, my_ID)
    delay = self.receive_pong(sock, my_ID, self.timeout)
    sock.close()
    return delay

def ping(self):
    """
    Run the ping process
    """
    for i in xrange(self.count):
        print "Ping to %s..." % self.target_host,
        try:
            delay = self.ping_once()
        except socket.gaierror, e:
            print "Ping failed. (socket error: '%s')" % e[1]
            break

        if delay == None:
            print "Ping failed. (timeout within %ssec.)" % self.timeout

```

```

else:
    delay = delay * 1000
    print "Get pong in %.4fms" % delay

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Python ping')
    parser.add_argument('--target-host', action="store",
dest="target_host", required=True)
    given_args = parser.parse_args()
    target_host = given_args.target_host
    pinger = Pinger(target_host=target_host)
    pinger.ping()

```

How many function the code have? Which is the command for running the script (Target host is WWW.GOOGLE.COM)?

Is there any error?

What is the solution? Which is the output?

Use the script using your own IP address? Which is the output?

- **Exercise 4.6:** Pinging hosts on the network with ICMP using pc resources
Create python scrip using the syntax below (save as ping_subprocess.py):

```

Created on 4 Jul 2016

@author: e05975
'''
import subprocess
import shlex

command_line = "ping -c 1 10.0.1.135"

if __name__ == '__main__':
    args = shlex.split(command_line)
    try:

subprocess.check_call(args,stdout=subprocess.PIPE,stderr=subprocess.PIPE)
        print ("Your pc is up!")
    except subprocess.CalledProcessError:
        print ("Failed to get ping.")

```

Add the IP address of your PC in the code and run the script, which is the output?

Test the script with the IP of you classmate?

What is the role of subprocess?

- **Exercise 4.7:** Scanning the broadcast of packets
Create python scrip using the syntax below (save as broadcast_scanning.py):

```

from scapy import all
from scapy.layers.inet import sr, srp, IP, UDP, ICMP, TCP, ARP, Ether,
sniff
captured_data = dict()

```

```

END_PORT = 1000

def monitor_packet(pkt):
    if IP in pkt:
        if not captured_data.has_key(pkt[IP].src):
            captured_data[pkt[IP].src] = []

    if TCP in pkt:
        if pkt[TCP].sport <= END_PORT:
            if not str(pkt[TCP].sport) in captured_data[pkt[IP].src]:
                captured_data[pkt[IP].src].append(str(pkt[TCP].sport))

    os.system('clear')
    ip_list = sorted(captured_data.keys())
    for key in ip_list:
        ports = ', '.join(captured_data[key])
        if len(captured_data[key]) == 0:
            print('%s' % key)
        else:
            print('%s (%s)' % (key, ports))

if __name__ == '__main__':
    sniff(prn=monitor_packet, store=0)

```

Run the script, which is the output?

- **Exercise 4.8:** Sniffing packets on your network

Tcpdump is a common packet analyzer that runs under the command line. It allows the user to display TCP/IP and other packets being transmitted or received over a network to which the computer is attached. Distributed under the BSD license,[3] tcpdump is free software.

- Open a linux terminal and check the usage of tcpdump using the command line `tcpdump -help`
- Using tcpdump get the traffic present in the Ethernet interface of your pc (10 packet only), which is the command line?
- Using the subprocess write a program for sniffing 1 packet of the Ethernet interface? (Save as `packet_sniffer.py`).

- **Exercise 4.9:** Performing a basic Telnet

Create python scrip using the syntax below (save as `echo_client.py`):

```

#!/usr/bin/env python

import socket

TCP_IP = '127.0.0.1'
TCP_PORT = 62
BUFFER_SIZE = 20 # Normally 1024, but we want fast response

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind((TCP_IP, TCP_PORT))
s.listen(1)

conn, addr = s.accept()
print('Connection address:', addr)

```

```
while 1:
    data = conn.recv(BUFFER_SIZE)
    if not data: break
    print ("received data:", data)
    conn.send(data) # echo
conn.close()
```

Add the IP address of your PC in the code and run the script, which is the output?

- Open a linux terminal and execute “telnet ‘IP PC’ port”, what happen with the server console?
- Test the script with the IP of you classmate?

5. Questions

- **Question 5.1:** Explain in your own words what is a network interface?
- **Question 5.2:** How many network interface usually you find in your pc?
- **Question 5.3:** Explain why you sniffing the network interface? Give examples?
- **Question 5.4:** Explain why it is relevant to communicate using sockets?
- **Question 5.5:** Provide a script that allows you to get information from internet? For example reading new? Reading Wikipedia? Reading Google map?

Lab 4: SDN Controllers and Mininet

1. Objectives

The objective of the lab 4 is to:

- Install and use traffic generators as powerful tools for testing network performance.
- Install and configure SDN Controller
- Install and understand how the mininet simulator works
- Implement and run basic examples for understanding the role of the controller and how it interact with mininet

2. Theory

2.1. Traffic Generator:

What is iPerf?: iPerf is a tool for active measurements of the maximum achievable bandwidth on IP networks. It supports tuning of various parameters related to timing, buffers and protocols (TCP, UDP, SCTP with IPv4 and IPv6). For each test it reports the bandwidth, loss, and other parameters.

iPerf features

- TCP and [SCTP](#)
 - Measure bandwidth
 - Report MSS/MTU size and observed read sizes.
 - Support for TCP window size via socket buffers.
- UDP
 - Client can create UDP streams of specified bandwidth.
 - Measure packet loss
 - Measure [delay jitter](#)
 - Multicast capable
- Cross-platform: Windows, Linux, Android, MacOS X, FreeBSD, OpenBSD, NetBSD, [VxWorks](#), Solaris,...
- Client and server can have multiple simultaneous connections (-P option).
- Server handles multiple connections, rather than quitting after a single test.
- Can run for specified time (-t option), rather than a set amount of data to transfer (-n or -k option).
- Print periodic, intermediate bandwidth, jitter, and loss reports at specified intervals (-i option).
- Run the server as a daemon (-D option)
- Use representative streams to test out how link layer compression affects your achievable bandwidth (-F option).

2.2. Software Defined Networking:

Software-defined networking was pioneered between 2008 and 2011 by work done at Stanford University and the Nicira Company (now part of VMware). The basic premise behind SDN is

that by separating control of network functions from hardware devices, administrators acquire more power to route and direct traffic in response to changing requirements.

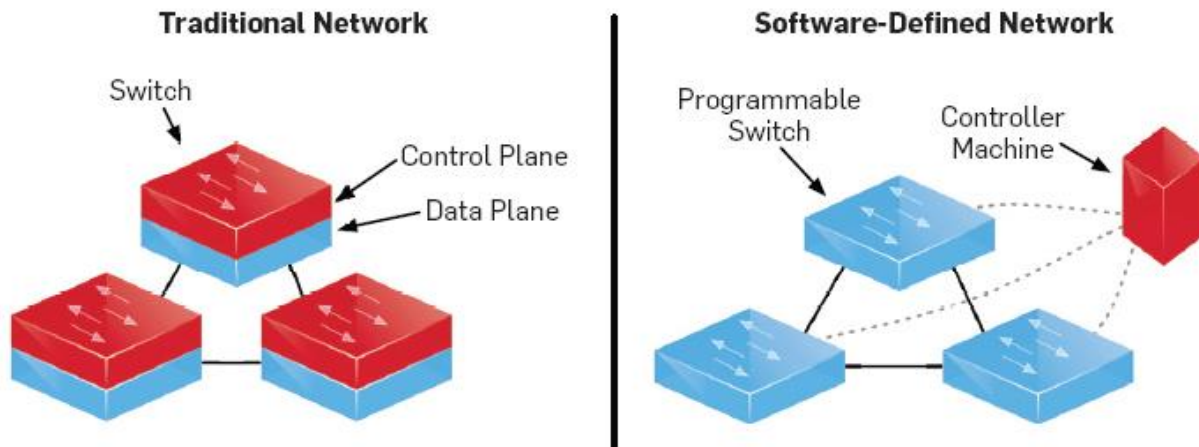


Figure 2-1. Traditional networks versus software defined networks.

Software-Defined vs. Traditional Networking: The key difference between traditional and software-defined networking is how SDNs handle data packets. In a traditional network, the way a switch handles an incoming data packet is written into its firmware. Most switches — particularly those used in commercial data centers rather than enterprise environments — respond to and route all packets the same way. SDN provides admins with granular control over the way switches handle data, giving them the ability to automatically prioritize or block certain types of packets. This, in turn, allows for greater efficiency without the need to invest in expensive, application-specific network switches.

Benefits of Software-Defined Networking: There are several benefits to the more advanced level of control afforded by implementing SND in a multi-tenant network environment:

- **Automation:** SND allows for automation of complex operational tasks that make networks faster, more efficient and easier to manage.
- **Increased uptime:** SDN has proven effective in reducing deployment and configuration errors that can lead to service disruptions.
- **Less drain on resources:** SDN gives administrators control over how their routers and switches operate from a single, virtual workflow. This frees up key staff to focus on more important tasks.
- **Better visibility:** With SDN, system administrator's gain improved visibility into overall network function, allowing them to allocate resources more effectively.
- **Cost savings:** SND can lead to significant overall costs savings. It also reduces the amount of spending required on infrastructure by allowing data centers to get the most use of their existing devices.

2.2.1. Controller:

OVS-testcontroller is a simple OpenFlow controller that manages any number of switches over the OpenFlow protocol, causing them to function as L2 MAC-learning switches or hubs. It is suitable for initial testing of OpenFlow networks.

Ryu is a component-based software defined networking framework. Ryu provides software components with well-defined API that make it easy for developers to create new network management and control applications. Ryu supports various protocols for managing network devices, such as OpenFlow, Netconf, OF-config, etc. About OpenFlow, Ryu supports fully 1.0, 1.2, 1.3, 1.4, 1.5 and Nicira Extensions. All of the code is freely available under the Apache 2.0 license.

2.2.2. Mininet: Mininet creates a **realistic virtual network**, running **real kernel, switch and application code**, on a single machine (VM, cloud or native) Because you can easily interact with your network using the Mininet CLI (and API), customize it, share it with others, or deploy it on real hardware, Mininet is useful for development, teaching, and research. Mininet is also a great way to develop, share, and experiment with OpenFlow and Software-Defined Networking systems.

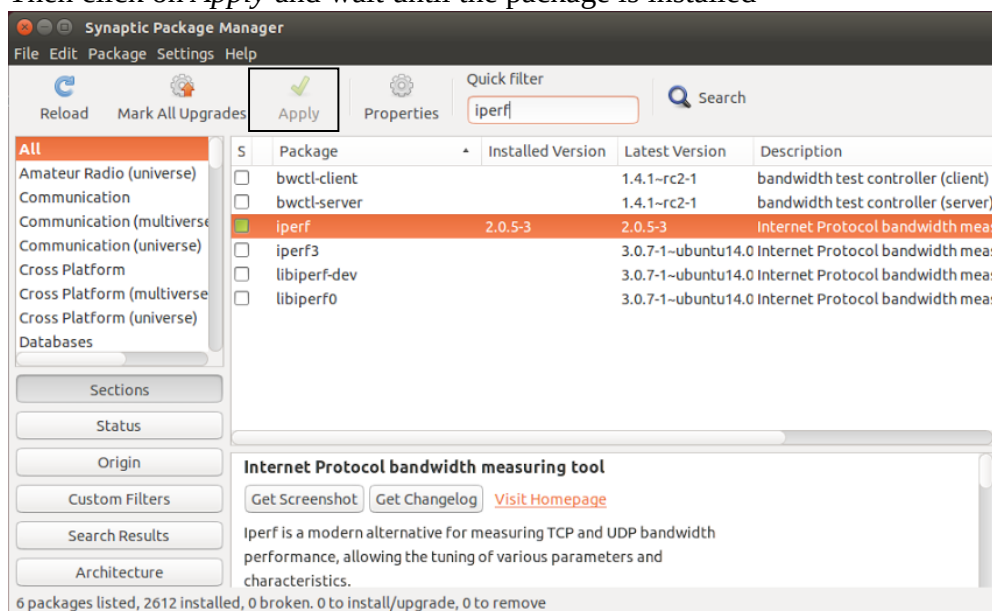
3. Methodology

TIP: For getting extra space in your USB-please use the following tips:

- Empty the trash
- Delete the Android related stuff
- Delete the extras for other courses
- Delete the already installed package sources

Install iperf

1. Open the *Synaptic Package Manager* (Navigator ->System-> *Synaptic Package Manager*)
2. Setup the proxy:
 - Click on *settings-> Preference -> Network*
 - Click on *manual proxy configuration*
 - HTTP and FTP Proxy: proxy.rmit.edu.au Port: 8080
3. Search for *Quick filter`iperf`*
4. Click on *Mark for installation*
5. Then click on *Apply* and wait until the package is installed



Install mininet

1. In *Synaptic Package*
2. Search for *Quick filter`mininet`*
3. Click on *Mark for installation*
4. Then click on *Apply* and wait until the package is installed

Install Controller

OVS controller:

1. In *Synaptic Package*
2. Search for *Quick filter`openvswitch-controller`*
3. Click on *Mark for installation*
4. Then click on *Apply* and wait until the package is installed

RYU controller:

NOTE: Installation valid for Internet access without Proxy

1. Run the command line: *sudo pip install ryu*
2. In some cases the system will not install the required packages by default, command line in this case is: ***sudo pip install ryu eventlet routes webob paramiko lxml netaddr oslo.config msgpack-python greenlet repoze.lru ecdsa six stevedore rfc3986 debtcollector***
3. Download the following packages:
 - Six package: <https://pypi.python.org/pypi/six/1.9.0>
 - Netaddr package: <https://pypi.python.org/pypi/netaddr#downloads>
 - Babel package: <https://pypi.python.org/pypi/Babel>
 - Six package: <https://pypi.python.org/pypi/six>
 - Oslo package: <https://pypi.python.org/pypi/oslo.i18n>
 - Stevedore package: <https://pypi.python.org/pypi/stevedore>
 - RYU package: <https://github.com/osrg/ryu>
4. Extract the tarball.
5. Change into the new directory that has been newly extracted.
6. Run the command line: *sudo python setup.py build*
7. Run the command line: *sudo python setup.py install*

4. Exercises

Section 4.1: Using traffic Generators

- **Exercise 4.1.1:** Open a Linux terminal, and execute the command line *iperf --help*. Provide four configuration options of *iperf*.
- **Exercise 4.1.2:** Open two Linux terminals, and configure terminal-1 as client (*iperf -c IPv4_server_address*) and terminal-2 as server (*iperf -s*). **Note: use the loopback address. Which are the statistics provided at the end of transmission?**
- **Exercise 4.1.3:** Open two Linux terminals, and configure terminal-1 as client and terminal-2 as server for exchanging UDP traffic, which are the command lines? Which are the statistics provided at the end of transmission? What is different from the statistics provided in exercise 4.1.1.
- **Exercise 4.1.4:** Open two Linux terminals, and configure terminal-1 as client and terminal-2 as server for exchanging UDP traffic, with:

- Packet length = 1000bytes
- Time = 20 seconds
- Bandwidth = 1Mbps
- Port = 9900

Which are the command lines?

Exercise 4.1.5: Open Wireshark (work with your partner for this task),

- Navigator -> Internet->Wireshark
- Capture all the packets in the Ethernet interface (Capture->ethx->apply)
- Student 1: Open one Linux terminal, and configure terminal-1 as server for exchanging UDP traffic for 10s in port 9900.
- Student 2: Open one Linux terminal, and configure terminal-1 as client (use IPv4 address of Student 1) for exchanging UDP traffic for 10s in port 9900 (use a packet length of 1280 bytes).
- Check the capture packets in Wireshark, how you can visualize the traffic exchanged with student 1?
- **Exercise 4.1.5:** Using iperf find the saturation point of the Ethernet segment (work with your partner for this task). Ask your tutor for help.

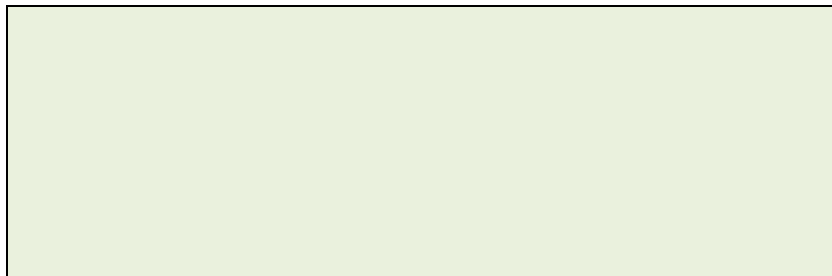
Section 4.2: Using Mininet (Note that if you run the mininet command without specifying a controller, it will use the ovsc controller, by default)

- **Exercise 4.2.1:** Open two Linux terminals, and execute the command line *ifconfig* in terminal-1. How many interfaces are present?

In terminal-2, execute the command line *sudo mn*, which is the output?

In terminal-1 execute the command line *ifconfig*. How many real and virtual interfaces are present now?

- **Exercise 4.2.2:** Interacting with mininet; in terminal-2, display the following command lines and explain what it does:
 - mininet> help
 - mininet> nodes
 - mininet> net
 - mininet> dump
 - mininet> h1 ifconfig -a
 - mininet> s1 ifconfig -a
 - mininet> h1 ping -c 5 h2
 - Draw the network topology that is created in mininet:



In terminal-1, display the following command line: *sudo ovs-vsctl show*, what is displayed?

Finish the test using `mininet>exit`

- **Exercise 4.2.3:** In terminal-2, display the following command line: `sudo mn --link tc,bw=10,delay=500ms`
 - `mininet> h1 ping -c 5 h2`, What happen with the link?
 - `mininet> h1 iperf -s -u &`
 - `mininet> h2 iperf -c IPv4_h1 -u`, Is there any packet loss?

Modify iperf for creating packet loss in the mininet network, which is the command line?

Section 4.3: Using and configuring your controller

- **Exercise 4.3.1:** Local controller: Mininet can be used with python scripts as well. The following script reproduce same architecture as before: (save as .py)

```
from mininet.cli import CLI
from mininet.net import Mininet
from mininet.node import OVSController

if '__main__' == __name__:
    net = Mininet(controller=OVSController )
    c0 = net.addController('c0', port=6633)
    s1 = net.addSwitch('s1')

    h1 = net.addHost('h1')
    h2 = net.addHost('h2')

    net.addLink(s1, h1)
    net.addLink(s1, h2)

    net.build()

    c0.start()

    s1.start([c0])

    net.startTerms()
    CLI(net)
    net.stop()
```

In terminal-1, run the script. The script will open four terminals; run the following command line in the terminals:

1. terminal-2 C0 (*ifconfig*), get the virtual interface and run (`tcpdump -i s1-ethx arp`)
2. terminal-3 S1 (*ifconfig*), get the virtual interface and run (`tcpdump -i s1-ethx arp`)
3. terminal-4 H1 (*ifconfig*), get the virtual interface and run (`tcpdump -i h1-ethx arp`)
4. terminal-5 H2 (*ifconfig*), get the virtual interface and run (`tcpdump -i h2-ethx arp`)

Terminal C0	Terminal S1
Terminal h1	Terminal h2

Example of Terminal-2, Terminal-3, Terminal-4, Terminal-5

In terminal-1, display the following command lines:

- mininet> nodes
- mininet> net
- mininet> dump
- mininet> h1 ping -c 5 h2

Which is the role of controller with ARP packets?

In terminal-1, display the following command line:

- mininet> h1 ping -c 5 h2

What happen this time? Why?

- **Exercise 4.3.1:** Remote controller: The following script reproduce same architecture as before using an external controller: (save as remote_controller.py)

```
from mininet.cli import CLI
from mininet.net import Mininet
from mininet.node import RemoteController

if '__main__' == __name__:
    net = Mininet(controller=RemoteController)
    c0 = net.addController('c0', port=6633)

    s1 = net.addSwitch('s1')

    h1 = net.addHost('h1')
    h2 = net.addHost('h2')

    net.addLink(s1, h1)
    net.addLink(s1, h2)

    net.build()

    c0.start()

    s1.start([c0])

    net.startTerms()
    CLI(net)
    net.stop()
```

The following scrip defines the behavior of the controller: (save as simple_switch_13.py, it is also available in blackboard)

```

from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from ryu.controller.handler import set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib.packet import packet
from ryu.lib.packet import ethernet

class SimpleSwitch13(app_manager.RyuApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(SimpleSwitch13, self).__init__(*args, **kwargs)
        self.mac_to_port = {}

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        datapath = ev.msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        # install table-miss flow entry
        #
        # We specify NO BUFFER to max_len of the output action due to
        # OVS bug. At this moment, if we specify a lesser number, e.g.,
        # 128, OVS will send Packet-In with invalid buffer_id and
        # truncated packet data. In that case, we cannot output packets
        # correctly.
        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                          ofproto.OFPCML_NO_BUFFER)]
        self.add_flow(datapath, 0, match, actions)

    def add_flow(self, datapath, priority, match, actions):
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]

        mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def _packet_in_handler(self, ev):
        msg = ev.msg
        datapath = msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        in_port = msg.match['in_port']

```



```

pkt = packet.Packet(msg.data)
eth = pkt.get_protocols(ethernet.ethernet)[0]

dst = eth.dst
src = eth.src

dpid = datapath.id
self.mac_to_port.setdefault(dpid, {})

self.logger.info("packet in %s %s %s %s", dpid, src, dst, in_port)

# learn a mac address to avoid FLOOD next time.
self.mac_to_port[dpid][src] = in_port

if dst in self.mac_to_port[dpid]:
    out_port = self.mac_to_port[dpid][dst]
else:
    out_port = ofproto.OFPP_FLOOD

actions = [parser.OFPACTIONOutput(out_port)]

# install a flow to avoid packet_in next time
if out_port != ofproto.OFPP_FLOOD:
    match = parser.OFPMATCH(in_port=in_port, eth_dst=dst)
    self.add_flow(datapath, 1, match, actions)

data = None
if msg.buffer_id == ofproto.OFP_NO_BUFFER:
    data = msg.data

out = parser.OFPPACKETOut(datapath=datapath, buffer_id=msg.buffer_id,
                          in_port=in_port, actions=actions, data=data)
datapath.send_msg(out)

```

In terminal-1, run remote_controller.py script (DO NOT RUN THE simple_switch_13.py at this point). The script open four Linux terminals, run the following command line in the terminals:

1. terminal-2 C0 (*ifconfig*), get the virtual interface and run (tcpdump -i s1-ethx arp)
2. terminal-3 S1 (*ifconfig*), get the virtual interface and run (tcpdump -i s1-ethx arp)
3. terminal-4 H1 (*ifconfig*), get the virtual interface and run (tcpdump -i h1-ethx arp)
4. terminal-5 H2 (*ifconfig*), get the virtual interface and run (tcpdump -i h2-ethx arp)

In terminal-1, display the following command lines:

- mininet> h1 ping -c 5 h2

What happen? Why?

Open another terminal-6 run *sudo ovs-vsctl set Bridge s1 protocols=OpenFlow13*

In terminal terminal-2 Controller run *ryu-manager ./simple_switch_13.py*

In terminal-1, display the following command lines:

- mininet> h1 ping -c 5 h2

What happen? Why?

5. Questions

Question 5.1: Explain how the traffic generators work?

Question 5.2: Provide another example of traffic generator and how it work.

Question 5.3: Which is the main difference between configuring UDP and TCP traffic?

Question 5.4: In your opinions which are the main advantages and disadvantages of working with mininet? Provide at least 3.

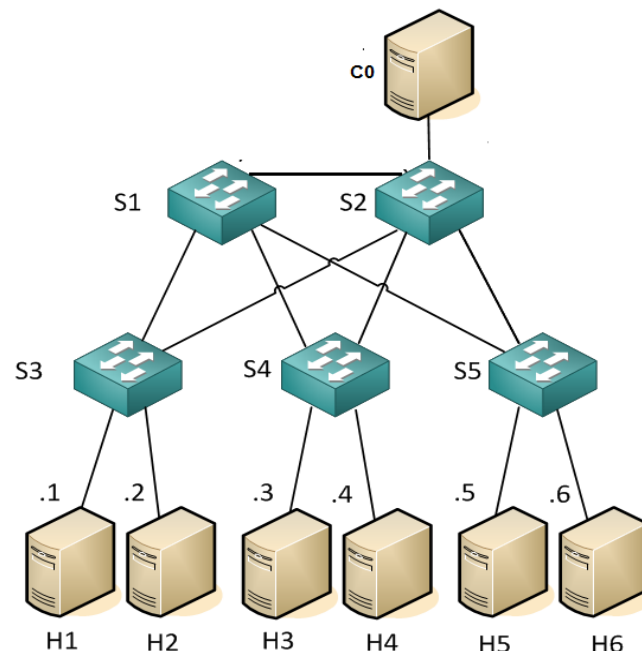
Question 5.5: Explain the architecture of Software defined Networks?

- Controller role:
- Switch role:
- Host role:

Question 5.6: What is the advantage of having a programmable Controller?

Question 5.7: Search for another Controller and provide the description and features.

Question 5.6: Provide the python code for emulation the following network.



6. References

- [1] Iperf Traffic generator. Available at: <https://iperf.fr> [Last access 15/08/2016]
- [2] RYU Controller. Available at: <https://osrg.github.io/ryu/> [Last access 15/08/2016]
- [3] Mininet Network Emulator. Available at: <http://mininet.org/overview/> [Last access 15/08/2016]

Lab 5: Zodiac OpenFlow Switch (Configure)

1. Objectives

The objective of the lab 5 is to:

- Configure and interact with Zodiac FX OpenFlow Switch.
- Exploring the Zodiac FX context.

2. Theory

2.1. Zodiac FX Switch:

Zodiac FX is the first OpenFlow switch designed to sit in a desk, not in a datacenter. Until now the power of Software Defined Networking (SDN) was only available to the administrators of large corporate networks. Even though there are numerous free or open source SDN controllers the one thing that was missing was a small, affordable OpenFlow switch. In this demo, we present Zodiac FX the world's smallest OpenFlow Software Defined Network Switch.

Zodiac FX Description: The Zodiac FX is a 4 port network development board designed for hobbyists, students, researchers, embedded developers or anyone who requires a low cost network development platform. Even though it was initially designed to allow affordable access to OpenFlow enabled hardware it's open source firmware it can be used in any number of other applications. By providing the firmware source code users are free to not only create their own versions but also use it as a basis for a completely different type of device. Some such applications may include: Router, Bridge, Load Balancer, Web server, VPN concentrator and many more. The main communication peripherals of Zodiac FX are sketched in Fig.1.

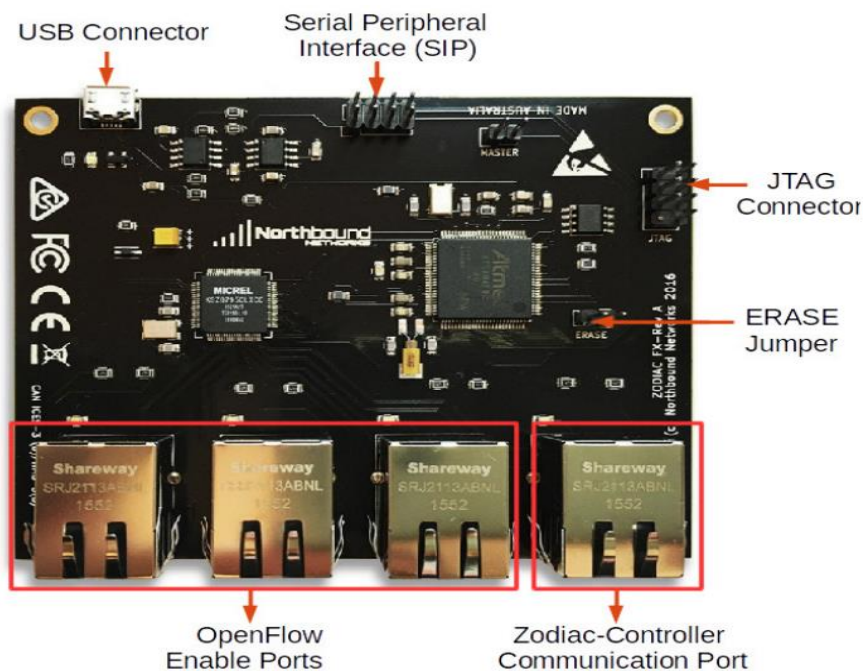


Figure 3-10. Communication peripherals of Zodiac FX.

2.2. IP addressing:

Static IP Addressing: With static IP addressing, addresses are assigned manually, and have to be provisioned carefully so that each device has its own address—with no overlap. When you connect a new device, you would have to select the "manual" configuration option and enter in the IP address, the subnet mask, the default gateway and the DNS server(s).

Dynamic Host Configuration Protocol (DHCP): DHCP takes all of the manual work out of IP addressing. Generally, the device that's at the "top" of your home network—whether it's a standalone firewall or a router/gateway device or your Control home controller—will provide DHCP by default as a service on the network. When DHCP is enabled, a new device connected to the network asks the DHCP server for an address, and the server assigns one from its pool of unused locations. The server itself tracks which addresses are used and which addresses are available, and keeps a record of which addresses have been assigned to the various devices. This ensures that addresses don't conflict with each other. However, it also means that, if a device goes offline, when it reconnects it may not have the same IP address it had before.

Mixing Configurations: It's entirely possible to mix static IP and DHCP addressing schemes. Since the default DHCP address range is between 100 and 149, you'll want to avoid all of the addresses between 192.168.1.100 and 192.168.1.149 when you're assigning static IP addresses. That leaves the ranges from 2-99 and from 150-254 wide open, which is usually plenty for most home networks.

2.2. Virtual Local Area Network (VLAN):

A VLAN is a group of devices on one or more LANs that are configured to communicate as if they were attached to the same wire, when in fact they are located on a number of different LAN segments. There are two main reasons for the development of VLANs:

1. the amount of broadcast traffic
2. increased security

Broadcast traffic increases in direct proportion to the number of stations in the LAN. The goal of the virtual LAN (VLAN) is the isolation of groups of users so that one group is not interrupted by the broadcast traffic of another. By segregating a group of devices to a particular VLAN, a switch will block broadcasts from devices in that VLAN to devices that are not in that VLAN instead of flooding it out every port. VLANs also have the benefit of added security by separating the network into distinct logical networks. Traffic in one VLAN is separated from another VLAN as if they were physically separate networks. If traffic is to pass from one VLAN to another, it must be routed.

Each VLAN is identified by a VLAN ID (VID), which is usually a number. They can reside on only a single switch, or they can be distributed throughout the entire network on each switch. Each VLAN is a broadcast domain. Each device in a VLAN, regardless of its physical location, can communicate directly with every other device in the same VLAN. However, they cannot communicate outside of the VLAN except through a router. A VLAN is usually created using physical ports.

3. Methodology

Zodiac FX Command Line Interface (Z-CLI): The Zodiac CLI provides the ability to configure setting and monitor the operation of the Zodiac FX. To simplify operations the CLI uses the concept of a context's, this limits the available commands to only those available in the currently selected context. There are currently four available contexts: Base, Config, OpenFlow

and Debug. To enter the required context simply type the name of the context on the command line while at the base level. The return to the base level type exit. The current context is shown in bracket between the device name and the prompt. The following sections describe the commands available within each context; please note that all commands are lower-case only.

1. **Base Functionalities:** The following commands are available in this context:

- **config** Enter the config's context.
- **openflow** Enter the OpenFlow's context.
- **debug** Enter the debug's context.
- **show status** Displays the current device status.
- **show ports** Displays information about each Ethernet port including state, VLAN membership and traffic statistics.
- **show version** Display the firmware version.
- **help** Display a list of available commands.

2. **Config Functionalities:** The following commands are available in this context:

- **save** Saves the current configuration to non-volatile memory.
- **show config** Display the current device configuration.
- **show vlans** Displays a list of the currently configured VLANs.
- **set name <name>** Sets the device name. Maximum of 16 characters, entries will be truncated.
- **set mac-address <mac address>** Sets the MAC address of the device. The MAC address assigned to the device is located on a label on the underside of the device.
- **set ip-address <ip address>** Sets the device IP address
- **set netmask <netmask>** Set the device netmask
- **set gateway <ip address>** Sets the default gateway of the device
- **set of-controller <ip address>** Sets the IP address the OpenFlow controller
- **set of-port <tcp port>** Sets the TCP port of the OpenFlow Controller
- **set of-version <version>** Sets the device to only connect to an controller using the OpenFlow version specified. A value of 0 disables this function and allows the device to negotiate the version.
- **add vlan <vlan id> <vlan name>** Creates a new vlan. Valid IDs are 1-4096 and names must be less than 16 characters.
- **delete vlan <vlan id>** Deletes an existing vlan.
- **set vlan-type <vlan id> <type>** Set the vlan to either openflow or native.
- **add vlan-port <vlan id> <port>** Assigns a ethernet port to the designated vlan. A port can only be a member of one vlan.
- **delete vlan-port <port>** Remove the named Ethernet port from a vlan.
- **factory reset** Configures and saves the configuration back to the factory test configuration.
- **exit** Return the context back the base level.

3. **OpenFlow Functionalities:** The following commands are available in this context:

- **show status** Displays the OpenFlow status.
- **show flows** Displays a list of the currently installed flows.

- **enable** Enables the OpenFlow functionality.
- **disable** Disables the OpenFlow functionality.
- **clean flows** Disabling OpenFlow will clear the flow tables and
- **exit** Return the context back the base level.

4. **Debug Functionalities:** The following commands are available in this context:
- **read register** Display the value of the KSZ8795 register.
 - **write register < value>** Writes the value into the defined KSZ8795 register.
 - **exit** Return the context back the base level

4. Exercises

Section 4.1: Connecting to Zodiac (Using default settings). **Note:** Zodiac device and USB-Mini-USB cable will be provided by the tutor during the lab (one kit per student). Please bring an

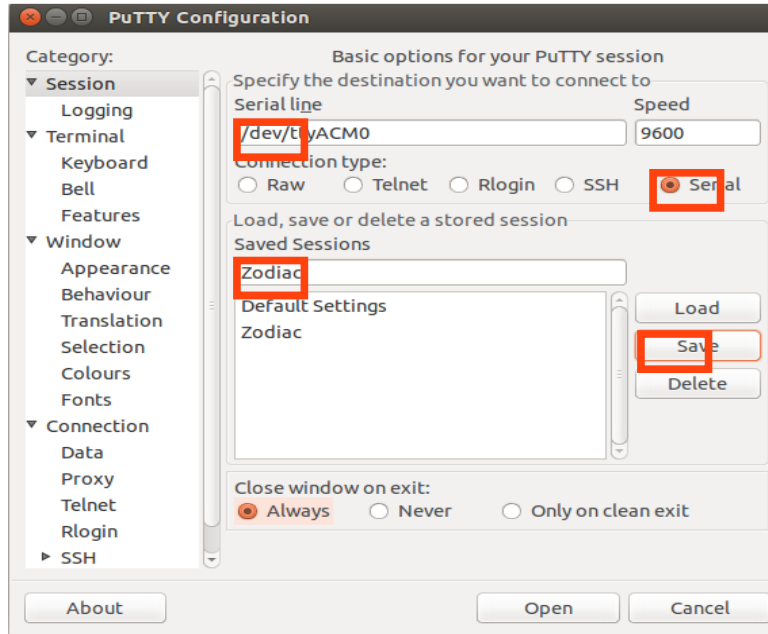
- **Exercise 4.1.1:** Using the USB-MiniUSB cable connect to Zodiac FX.



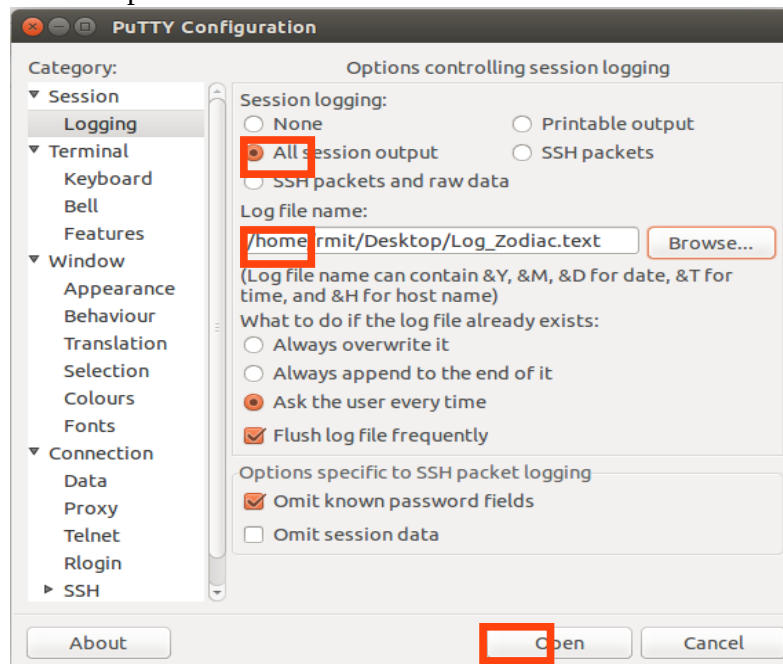
- **Exercise 4.1.2:** In a terminal check if the device is detected using the following the command lines
 - a. Run `cd /dev`
 - b. Run `ls` and look for the `ttyACMx` (where `x` represent the number that your Linux OS give to the device.)

```
rmit@rmit-HP-EliteBook-Folio-9470m:/dev$ ls
autofs      mem          sda6         tty3          tty62         ttyS7
block       net          sdb          tty30         tty63         ttyS8
bsg         network_latency serial        tty31         tty7          ttyS9
btrfs-control network_throughput sg0          tty32         tty8          uhid
bus         null         sg1          tty33         tty9          uinput
char        port         shm          tty34         ttyA:M0       urandom
console     ppp         snapshot     tty35         ttyprintk     v4l
core        psaux       snd          tty36         ttyS0         vboxdrv
```

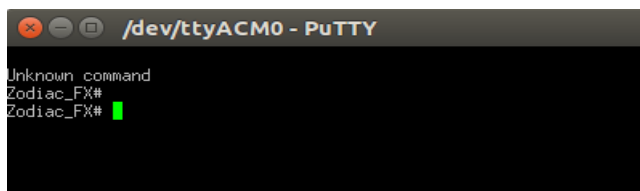
- **Exercise 4.1.3:** Open Putty Software: run the command `sudo putty` and configure it using the following procedure:
 - Configure the communication as show in the figure:
 - Serial Line: `/dev/ttyACMx`
 - Tick serial
 - Provide a name example Zodiac
 - Save the session



Configure the logs as show in the figure:
 Tick All the session output
 Provide a name and localization for the log
 Open the session



The following CLI will open (Z-CLI):



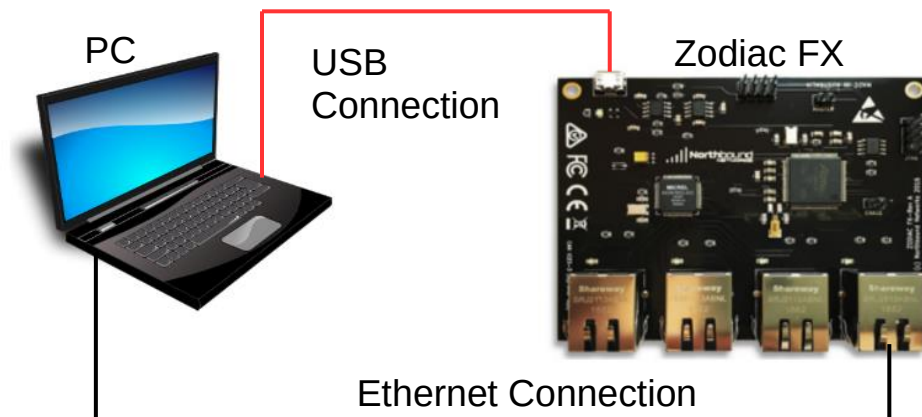
- **Exercise 4.1.4:** Using the Base Functionalities command lines in Z-CLI, check the information about each Ethernet port. Which is the status of the port? Which ports are OpenFlow? Are all the ports in the same VLAN?
- **Exercise 4.1.5:** Using the OpenFlow Functionalities command lines in Z-CLI, check the information about flows? Is there any flow? If not, why?
- **Exercise 4.1.6:** Using the Config Functionalities command lines in Z-CLI, check the Zodiac FX configuration, complete the following information:

1. MAC Address:
2. IP Address:
3. Netmask:
4. Gateway:
5. OpenFlow Controller:
6. OpenFlow Port:
7. Openflow Status:

Is the MAC address equal to the one provided in the hardware?

Display the status of the VLANs, in which VLAN the controller should be connected?

- **Exercise 4.1.6:** Connect the Ethernet cable between the Zodiac FX native port and the lab-PC as show in the following figure.



Show the status of the ports again using Z-CLI (same as **Exercise 4.1.4**), is the status of port 4 changed?

In the lab PC open a terminal and ping the zodiac device (ping -c 5 10.0.1.99), does the ping works? If not, why?

- **Exercise 4.1.7:** Configuring lab PC to communicate with Zodiac FX. In order to establish the communication between Zodiac FX and Lab-PC using native port, Lab-PC has to be configured as Controller. Therefore, configure lab-PC with the following static IP address:

1. IP Address: 10.0.1.8
2. Netmask: 255.255.255.0
3. Gateway: 10.0.1.1

In the lab PC open a terminal, ping the zodiac device again (ping -c 5 10.0.1.99), does the ping works? If yes, why?

Section 4.2: Changing Zodiac configuration.

- **Exercise 4.2.1:** Using the Config Functionalities command lines in Z-CLI, set Zodiac IP address to 10.0.1.(10+x), where x is the number of your Zodiac FX (check the box).

Save the configuration in the non-volatile memory.

- **Exercise 4.2.2:** In the lab PC open a terminal, ping the zodiac device again using the new IP address, does the ping work?

- **Exercise 4.2.1:** Using the Config Functionalities command lines in Z-CLI, create a new vlan the name SDN_lab and type openflow (Vlan-ID=300). Then add the port 3 to the new vlan. Save the configuration in the non-volatile memory.

When done display *show vlans* and *show ports* and ask your tutor for marking it.

5. Questions

Question 5.1: Explain the difference between the Native and OpenFlow ports?

Question 5.2: Why we cannot use dynamic IP between zodiac and lab-PC?

Question 5.3: What is the difference between OpenFlow and non-OpenFlow switch (apart from the support of openflow)

Question 5.4: Provide other examples of commercial OpenFlow switches.

6. References

- [1] Iperf Traffic generator. Available at: <https://iperf.fr> [Last access 15/08/2016]
- [2] RYU Controller. Available at: <https://osrg.github.io/ryu/> [Last access 15/08/2016]
- [3] Zodiac FX, Northbound Networks Pty. Ltd., 2016. [Online]. Available: <http://northboundnetworks.com/collections/zodiac-fx/products/zodiac-fx>. [Accessed: 16-May-2016].

Lab 6: OpenFlow Protocol

1. Objectives

The objective of the lab 6 is to:

- Understand the working principles of OpenFlow protocol.
- Configure a basic Software Defined Network for end-to-end communications.
- Understand the difference between interacting with real and virtual networks.

2. Theory

This specification covers the components and the basic functions of the switch, and the OpenFlow protocol to manage an OpenFlow switch from a remote controller.

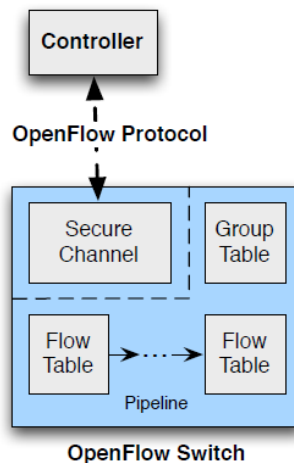


Figure 2-1. Main components of an OpenFlow switch.

2.1 Switch Components

An OpenFlow Switch consists of one or more flow tables and a group table, which perform packet lookups and forwarding, and an OpenFlow channel to an external controller (Figure 2-1). The switch communicates with the controller and the controller manages the switch via the OpenFlow protocol.

Using the OpenFlow protocol, the controller can add, update, and delete flow entries in flow tables, both reactively (in response to packets) and proactively. Each flow table in the switch contains a set of flow entries; each flow entry consists of match fields, counters, and a set of instructions to apply to matching packets.

Matching starts at the first flow table and may continue to additional flow tables. Flow entries match packets in priority order, with the first matching entry in each table being used. If a matching entry is found, the instructions associated with the specific flow entry are executed. If no match is found in a flow table, the outcome depends on configuration of the table-miss flow entry: for example, the packet may be forwarded to the controller over the OpenFlow channel, dropped, or may continue to the next flow table.

Instructions associated with each flow entry either contain actions or modify pipeline processing. Actions included in instructions describe packet forwarding, packet modification and group table processing. Pipeline processing instructions allow packets to be sent to subsequent tables for further processing and allow information, in the form of metadata, to be communicated between tables. Table pipeline processing stops when the instruction set associated with a matching flow entry does not specify a next table; at this point the packet is usually modified and forwarded.

Flow entries may forward to a port. This is usually a physical port, but it may also be a logical port defined by the switch or a reserved port defined by this specification. Reserved ports may specify generic forwarding actions such as sending to the controller, flooding, or forwarding using non-OpenFlow methods, such as *normal switch processing* while switch-defined logical ports may specify link aggregation groups, tunnels or loopback interfaces.

Actions associated with flow entries may also direct packets to a group, which specifies additional processing. Groups represent sets of actions for flooding, as well as more complex forwarding semantics (e.g. multipath, fast reroute, and link aggregation). As a general layer of indirection, groups also enable multiple flow entries to forward to a single identifier (e.g. IP forwarding to a common next hop). This abstraction allows common output actions across flow entries to be changed efficiently.

The group table contains group entries; each group entry contains a list of action buckets with specific semantics dependent on group type. The actions in one or more action buckets are applied to packets sent to the group.

Switch designers are free to implement the internals in any way convenient, provided that correct match and instruction semantics are preserved. For example, while a flow entry may use an all group to forward to multiple ports, a switch designer may choose to implement this as a single bitmask within the hardware forwarding table. Another example is matching; the pipeline exposed by an OpenFlow switch may be physically implemented with a different number of hardware tables.

2.2 OpenFlow specification terms

This section describes key OpenFlow specification terms:

- **Byte:** an 8-bit octet.
- **Packet:** an Ethernet frame, including header and payload.
- **Port:** where packets enter and exit the OpenFlow pipeline. May be a physical port, a logical port defined by the switch, or a reserved port defined by the OpenFlow protocol.
- **Pipeline:** the set of linked flow tables that provide matching, forwarding, and packet modifications in an OpenFlow switch.
- **Flow Table:** A stage of the pipeline, contains flow entries.
- **Flow Entry:** an element in a flow table used to match and process packets. It contains a set of match fields for matching packets, a priority for matching precedence, a set of counters to track packets, and a set of instructions to apply.
- **Match Field:** a field against which a packet is matched, including packet headers, the ingress port, and the metadata value. A match field may be wildcarded (match any value) and in some cases bitmasked.

- **Metadata:** a maskable register value that is used to carry information from one table to the next.
- **Instruction:** Instructions are attached to a flow entry and describe the OpenFlow processing that happen when a packet matches the flow entry. An instruction either modifies pipeline processing, such as direct the packet to another flow table, or contains a set of actions to add to the action set, or contains a list of actions to apply immediately to the packet.
- **Action:** an operation that forwards the packet to a port or modifies the packet, such as decrementing the TTL field. Actions may be specified as part of the instruction set associated with a flow entry or in an action bucket associated with a group entry. Actions may be accumulated in the Action Set of the packet or applied immediately to the packet.
- **Action Set:** a set of actions associated with the packet that are accumulated while the packet is processed by each table and that are executed when the instruction set instructs the packet to exit the processing pipeline.
- **Group:** a list of action buckets and some means of choosing one or more of those buckets to apply on a per-packet basis.
- **Action Bucket:** a set of actions and associated parameters, defined for groups.
- **Tag:** a header that can be inserted or removed from a packet via push and pop actions.
- **Outermost Tag:** the tag that appears closest to the beginning of a packet.
- **Controller:** An entity interacting with the OpenFlow switches using the OpenFlow protocol.
- **Meter:** a switch element that can measure and control the rate of packets. The meter trigger a meter band if the packet rate or byte rate passing through the meter exceed a predefined threshold. If the meter band drops the packet, it is called a Rate Limiter.

3. Methodology

This lab is designed for working on group of three students. In this activity students will learn how a switch hub works and how it is implemented using OpenFlow.

3.1 Traditional Switching Hub

Switching hubs have a variety of functions. Here, we take a look at a switching hub having the following simple functions.

- Learns the MAC address of the host connected to a port and retains it in the MAC address table.
- When receiving packets addressed to a host already learned, transfers them to the port connected to the host.
- When receiving packets addressed to an unknown host, performs flooding.

3.2 Switching Hub by OpenFlow

OpenFlow switches can perform the following by receiving instructions from OpenFlow controllers such as Ryu:

- Rewrites the address of received packets or transfers the packets from the specified port.
- Transfers the received packets to the controller (Packet-In).
- Transfers the packets forwarded by the controller from the specified port (Packet-Out).

It is possible to achieve a switching hub having those functions combined.

First of all, you need to use the Packet-In function to learn MAC addresses. The controller can use the Packet-In function to receive packets from the switch. The switch analyzes the received packets to learn the MAC address of the host and information about the connected port.

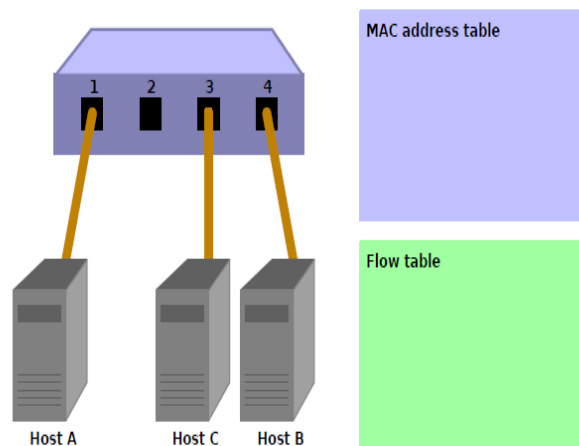
After learning, the switch transfers the received packets. The switch investigates whether the destination MAC address of the packets belong to the learned host. Depending on the investigation results, the switch performs the following processing.

- If the host is already a learned host ... Uses the Packet-Out function to transfer the packets from the connected port.
- If the host is unknown host ... Use the Packet-Out function to perform flooding.

The following explains the above operation in a step-by-step way using figures.

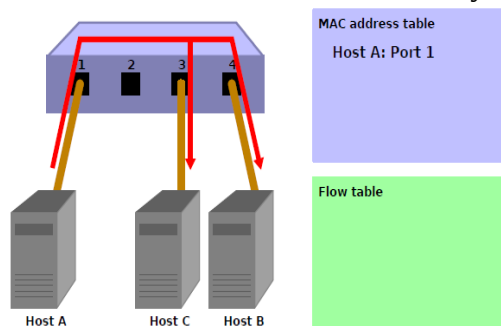
1. Initial status

- This is the initial status where the flow table is empty.
- Assuming host A is connected to port 1, host B to port 4, and host C to port 3.



2. Host A -> Host B

- When packets are sent from host A to host B, a Packet-In message is sent and the MAC address of host A is learned by port 1. Because the port for host B has not been found, the packets are flooded and are received by host B and host C.



Packet-In:

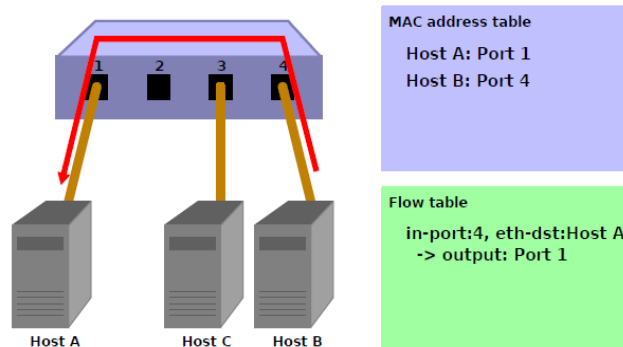
```
in-port: 1
eth-dst: Host B
eth-src: Host A
```

Packet-Out:

```
action: OUTPUT:Flooding
```

3. Host B -> Host A

- When the packets are returned from host B to host A, an entry is added to the flow table and also the packets are transferred to port 1. For that reason, the packets are not received by host C.



Packet-In:

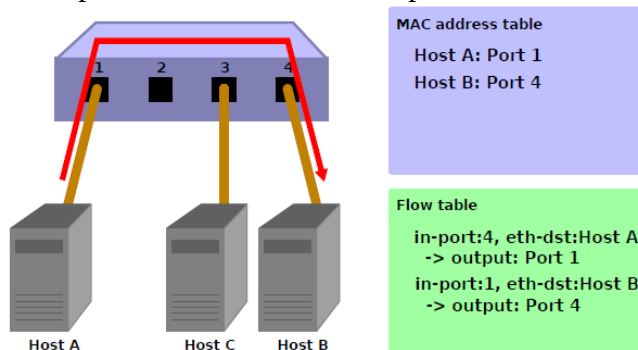
```
in-port: 4  
eth-dst: Host A  
eth-src: Host B
```

Packet-Out:

```
action: OUTPUT:Port 1
```

4. Host A -> Host B

- Again, when packets are sent from host A to host B, an entry is added to the flow table and also the packets are transferred to port 4.



Packet-In:

```
in-port: 1  
eth-dst: Host B  
eth-src: Host A
```

Packet-Out:

```
action: OUTPUT:Port 4
```

4. Exercises

Section 4.1: Using Graphical interface of the RYU controller. The `ryu.app.gui_topology` provides topology visualization.

Exercise 4.1.1: Run mininet in terminal-1 using the following script (save as remote_controller_mininet_tree.py). The scrip will create a tree topology.

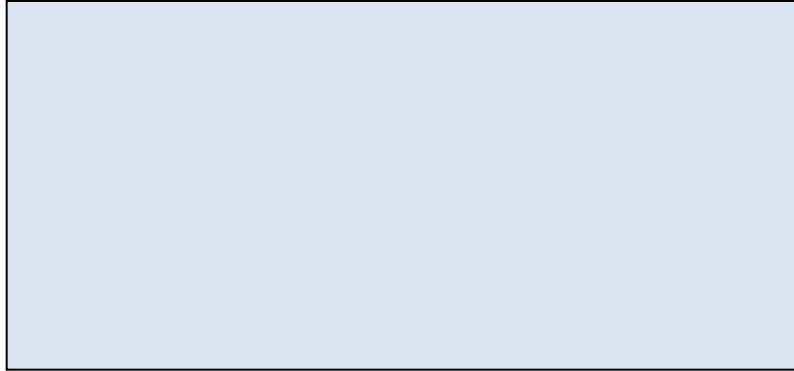
Exercise 4.1.2: Follow the following procedure:

```
from mininet.node import CPULimitedHost
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.log import setLogLevel, info
from mininet.node import RemoteController
from mininet.cli import CLI
from mininet.link import TCLink
class GenericTree(Topo):
    """Simple topology example."""
    def __init__( self, depth=1, fanout=2 ):
        # Numbering: h1..N, s1..M
        Topo.__init__(self)
        self.hostNum = 1
        self.switchNum = 1
        # Build topology
        self.addTree(depth, fanout)
    def addTree( self, depth, fanout ):
        """Add a subtree starting with node n.
        returns: last node added"""
        isSwitch = depth > 0
        if isSwitch:
            node = self.addSwitch( 's%s' % self.switchNum )
            self.switchNum += 1
            for _ in range( fanout ):
                child = self.addTree( depth - 1, fanout )
                self.addLink( node, child )
        else:
            node = self.addHost( 'h%s' % self.hostNum )
            self.hostNum += 1
        return node
def run():
    c = RemoteController('c', '0.0.0.0', 6633)
    # Change the args of GenericTree() to your desired values. You could
    even get them from command line.
    net = Mininet(topo=GenericTree(depth=3, fanout=2), host=CPULimitedHost,
    controller=None)
    net.addController(c)
    net.start()
    # installStaticFlows( net )
    CLI(net)
    net.stop()
    # if the script is run directly (sudo custom/optical.py):
if __name__ == '__main__':
    setLogLevel('info')
    run()
```

1. Move the ryu-master folder (which contains the ryu code) to the desktop. Download it from (<https://github.com/osrg/ryu>) in case you do not have it.
2. Then make sure that you have internet connection on Mozilla.

3. Go to /home/user/Desktop/ryu-master/ryu/app/gui_topology/html/
4. Open index.html using Mozilla (visualize a square) and close it.
5. Configure Mozilla with no proxy.
6. Run RYU GUI application in Terminal-1:
`sudo python /usr/local/bin/ryu run --observe-link /home/user/Desktop/ryu-master/ryu/app/gui_topology/gui_topology.py`
7. Access `http://<ip address of ryu host>:8080` with your web browser.
 - Example: <http://0.0.0.0:8080/>

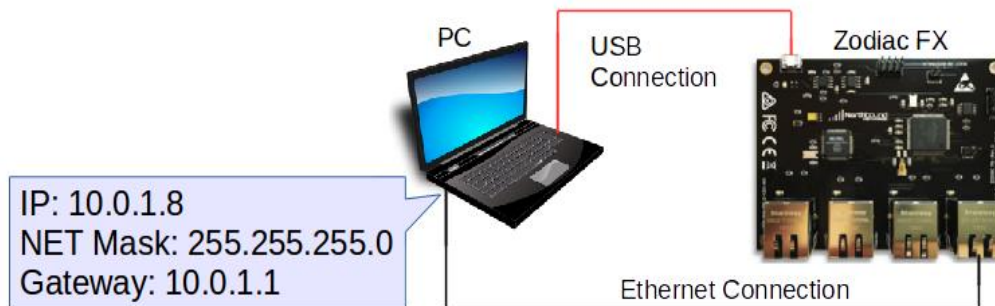
Exercise 4.1.3: Which is the topology displayed by the RYU GUI? Are the hosts displayed in the RYU GUI? Are the controller displayed?



Exercise 4.1.4: Using the script, create other two simple topologies?
Exit mininet

Section 4.2: Real network

Exercise 4.2.1: Connect Zodiac FX to the lab PC using the same configuration and topology of lab 5 as show in the figure.



Ping zodiac using the command line `ping 10.0.1.Zodiac_IPV4`. If the ping is not working, it means that the setup is wrong.

Fix the configuration until get the ping.

Exercise 4.2.2: Is Zodiac in the topology displayed by the RYU GUI? Is zodiac connected with the virtual network?

Exit mininet.

Exercise 4.2.3: (Work in group of three students for this section) Create a tree topology using the Zodiac FX switches. Is the topology display in the GUI? If not, why? Which port should use for interconnecting the switch? Explain.

Exercise 4.2.4: Create at least two different topologies using with Zodiac FX and show to your tutor, What you need to configure?

Finally, terminate RYU GUI by typing control-C in Terminal-1. Make sure that put Zodiac FX back to the following settings when finished the exercise.

Zodiac_FX(config)# show vlans

<i>VLAN ID</i>	<i>Name</i>	<i>Type</i>
<i>100</i>	<i>'Openflow'</i>	<i>OpenFlow</i>
<i>200</i>	<i>'Controller'</i>	<i>Native</i>
<i>300</i>	<i>'SDN_Lab'</i>	<i>OpenFlow</i>

Zodiac_FX# show vlan ports

Port 1

Status: DOWN

VLAN type: OpenFlow

VLAN ID: 100

Port 2

Status: DOWN

VLAN type: OpenFlow

VLAN ID: 100

Port 3

Status: DOWN

VLAN type: OpenFlow

VLAN ID: 300

Port 4

Status: UP

VLAN type: Native

VLAN ID: 200

Section 4.3: OpenFlow Protocol (Work individual)

Exercise 4.3.1: Connect to the Z-CLI, is there any flow?

Exercise 4.3.2: Run RYU manager (NOTE: this is another way to run the scrip provided in lab 4): *ryu-manager --verbose ryu.app.simple_switch_13*

Is there any flow now? What is the meaning of this flow? Which is the action?

Exercise 4.3.2: Work in groups of three students for this exercise,

1. **Student A:** Disconnect Zodiac FX and configure the PC with static IP address:
IP: 10.0.1.2

NET Mask: 255.255.255.0

Gateway: 10.0.1.1

2. **Student B:** Disconnect Zodiac FX and configure the PC with static IP address:

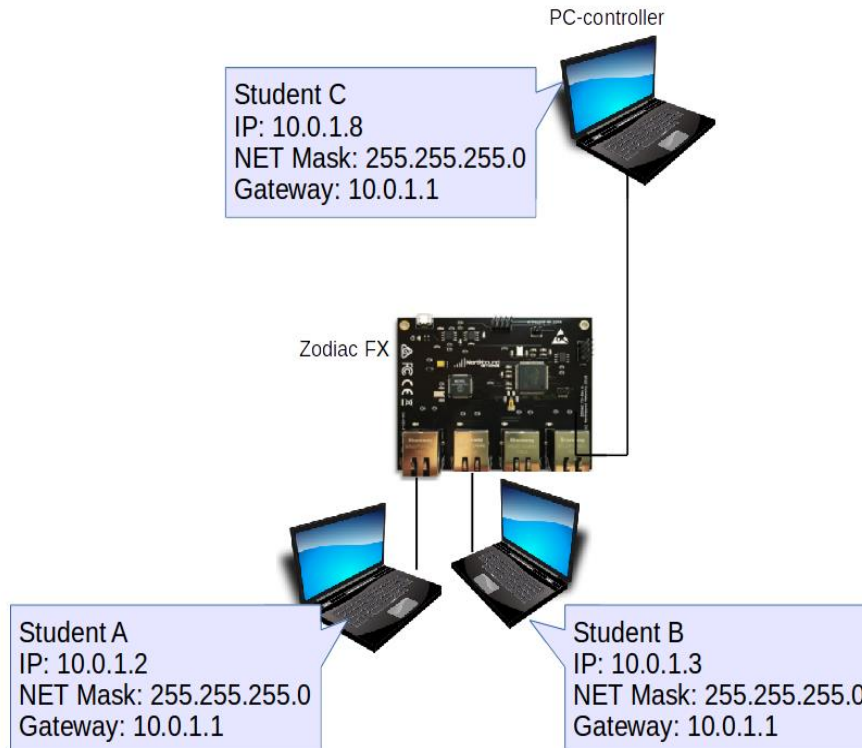
IP: 10.0.1.3

NET Mask: 255.255.255.0

Gateway: 10.0.1.1

3. **Student C:** Keep the same configuration and Zodiac FX.

Using the zodiac FX of student C, connect the PC of student A in port 1 and student B in port 2 as shown in the figure:



Exercise 4.3.3: From Student A PC ping Student B PC, is there any new flow? What is the role of the new flow? Which is the action?

Exercise 4.3.4: Check the status of zodiac port, is there any statistic?

Exercise 4.3.5: Clear the flows Zodiac-FX and perform the experiment again, this time capture the message exchanged between zodiac and controller using wireshark (activate it in controller PC). Identify the packet-in and packet-out messages. What is the format of the messages? Which fields the messages has?

Exercise 4.3.6: Switch the Ethernet cables of port 1 with port 2 and check the flow, is there any change? Ping the Student A PC from Student B PC what happen with the flows?

Exercise 4.3.7: **Switch** the cable of port 1 to port 3 and check the flow, is there any change? Ping the Student A PC from Student B PC, is the ping working? What happen with the flows?

Finally, make sure that put Zodiac FX back to the following settings when finished the exercise. Show the setting to your tutor.

```
Zodiac_FX(config)# show vlans
```

<i>VLAN ID</i>	<i>Name</i>	<i>Type</i>
100	'Openflow'	OpenFlow
200	'Controller'	Native
300	'SDN_Lab'	OpenFlow

```
Zodiac_FX# show vlan ports
```

Port 1

Status: DOWN

VLAN type: OpenFlow

VLAN ID: 100

Port 2

Status: DOWN

VLAN type: OpenFlow

VLAN ID: 100

Port 3

Status: DOWN

VLAN type: OpenFlow

VLAN ID: 300

Port 4

Status: UP

VLAN type: Native

VLAN ID: 200

5. Questions

Question 5.1: How the RYU GUI interface can be improved? Provide some ideas.

Question 5.2: Explain at least two the advantage and disadvantage of using mininet or Zodiac FX?

Question 5.3: Explain how the open flow tables are created?

Question 5.4: When Zodiac FX receive a packet, which functions an OpenFlow Switch performs:

Question 5.6: Provide your personal opinion about OpenFlow protocol? Focus on the functionalities and flexibility.

6. References

- [1] OpenFlow Protocol. Available at: <https://www.opennetworking.org/images/stories/downloads/sdn-resources/onf-specifications/openflow/openflow-spec-v1.3.1.pdf> [Last access 15/08/2016]
- [2] Iperf Traffic generator. Available at: <https://iperf.fr> [Last access 15/08/2016]

- [3] RYU Controller. Available at: <https://osrg.github.io/ryu/> [Last access 15/08/2016]
- [4] Zodiac FX, Northbound Networks Pty. Ltd., 2016. [Online]. Available: <http://northboundnetworks.com/collections/zodiac-fx/products/zodiac-fx>. [Accessed: 16-May-2016].

Lab 7: Controller Rest API

1. Objectives

The objective of the lab 7 is to:

- Understand the working principles of Controller Rest API.
- Understand the difference between proactive and reactive installation flows.

2. Theory

2.1 OpenFlow: Reactive versus Proactive

OpenFlow is still the only one wire protocol that has a reasonably good chance at becoming the de-facto open SDN southbound messaging standard. When using OpenFlow to populate tables in switches there are essentially three modes of operation:

- **Reactive Flow Instantiation:** When a new flow comes into the switch, the OpenFlow agent software on the switch does a lookup in the flow tables. If no match for the flow is found, the switch creates an OFP packet-in packet and sends it off to the controller for instructions. Reactive mode reacts to traffic, consults the OpenFlow controller and creates a rule in the flow table based on the instruction. This behavior was tested on previous lab.
- **Proactive Flow Instantiation:** Rather than reacting to a packet, an OpenFlow controller could populate the flow tables ahead of time for all traffic matches that could come into the switch. By pre-defining all of the flows and actions ahead of time in the switches flow tables, the packet-in event never occurs. The result is all packets are forwarded at line rate. Proactive OpenFlow flow tables eliminate any latency induced by consulting a controller on every flow. This behavior will be tested on this lab.
- **Hybrid flow instantiation:** A combination of both would allow for flexibility of reactive for particular sets a granular traffic control that while still preserving low-latency forwarding for the rest of the traffic.

2.2 Controller: REST API

- **Application program interface (API)** is an interface presented by software (such as a network operating system) that provides the capability to collect information from or make a change to an underlying set of resources.
- **APIs in the context of SDN:** In an open SDN model, a common interface discussed is the northbound interface (NBI). The NBI is the interface between software applications, such as operational support systems, and a centralized SDN controller. One of the common API technologies used at the northbound interface is the Representational State Transfer (REST) API. REST APIs use the HTTP/HTTPS protocol to execute common operations on resources represented by Uniform Resource Identifier (URI) strings. An application may use REST APIs to send an HTTP/HTTPS GET message via an SDN controller's IP address. That message would contain a URI string referencing the relevant network device and comprising an HTTP payload with a JSON header that has the proper parameters for a particular interface and statistic.

- **Datapath Identifier of Openflow Switch:** Each OpenFlow instance on a switch is identified by a Datapath Identifier. This is a 64 bit number determined as follows according to the OpenFlow specification: “The datapath_id field uniquely identifies a datapath. The lower 48 bits are intended for the switch MAC address, while the top 16 bits are up to the implementer. An example use of the top 16 bits would be a VLAN ID to distinguish multiple virtual switch instances on a single physical switch.”

3. Methodology

In this activity students will learn how to create flows using the Controller REST API.

Using REST APIs:

REST API can be used in different ways:

1. A tool to generate REST API calls:
 - ✓ The Chrome browser, for example, has multiple plug-ins to generate REST API messages. These include Postman and the Advanced REST Client.
 - ✓ Firefox has the RESTClient add-on for the same functionality.
2. Command-line interface, the curl utility may also be used.

Although the formatting of the REST API varies from one controller to another, the following items are common: URI string for the requested, HTTP method (e.g., GET, POST, PUT, and DELETE) and JSON/XML payload and/or parameters. The Ryu documentation provides examples illustrating how to send a valid REST API message.

RYU.APP.OFCTL_REST

(Information available at http://ryu.readthedocs.io/en/latest/app/ofctl_rest.html)

ryu.app.ofctl_rest provides REST APIs for retrieving the switch stats and updating the switch stats. This application helps to debug application and get various statistics. Valid actions are:

1. Retrieve the switch stats
 - ✓ Get all switches
 - ✓ Get the desc stats
 - ✓ Get all flows stats
 - ✓ Get flows stats filtered by fields
 - ✓ Get aggregate flow stats
 - ✓ Get aggregate flow stats filtered by fields
 - ✓ Get table stats
 - ✓ Get table features
 - ✓ Get ports stats
 - ✓ Get ports description
 - ✓ Get queues stats
 - ✓ Get queues config
 - ✓ Get queues description
 - ✓ Get groups stats
 - ✓ Get group description stats
 - ✓ Get group features stats
 - ✓ Get meters stats
 - ✓ Get meter config stats
 - ✓ Get meter description stats

- ✓ Get meter features stats
- 2. Update the switch stats
 - ✓ Add a flow entry
 - ✓ Modify all matching flow entries
 - ✓ Modify flow entry strictly
 - ✓ Delete all matching flow entries
 - ✓ Delete flow entry strictly
 - ✓ Delete all flow entries
 - ✓ Add a group entry
 - ✓ Modify a group entry
 - ✓ Delete a group entry
 - ✓ Modify the behavior of the port
 - ✓ Add a meter entry
 - ✓ Modify a meter entry
 - ✓ Delete a meter entry
 - ✓ Modify role
- 3. Support for experimenter multipart
 - ✓ Send a experimenter message
- 4. Reference: Description of Match and Actions
 - ✓ Description of Match on request messages
 - ✓ Description of Actions on request messages

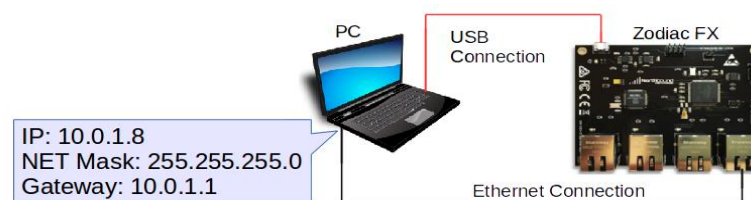
Installing curl:

1. Open the *Synaptic Package Manager* (Navigator -> System-> *Synaptic Package Manager*)
2. Setup the proxy:
 - Click on *settings-> Preference -> Network*
 - Click on *manual proxy configuration*
 - HTTP and FTP Proxy: proxy.rmit.edu.au Port: 8080
3. Search for *Quick filter `curl`*
4. Click on *Mark for installation*
5. Then click on *Apply* and wait until the package is installed

4. Exercises

Section 4.1: Using Rest API of RYU controller, perform the following exercises.

Exercise 4.1.1: Connect Zodiac FX to the lab PC using the same configuration and topology of lab 5 as show in the figure.



Ping zodiac using the command line `ping 10.0.1.Zodiac_IPV4`. If the ping is not working, it means that the setup is wrong.

Fix the configuration until get the ping.

Exercise 4.1.2: Run ryu controller including the two applications i) simple switch and 2) RYU.APP.OFCTL_REST using the following command line:

```
sudo ryu-manager ryu.app.simple_switch_13 ryu.app.ofctl_rest
```

Using ZODIAC_CLI check if there is any flow?

Exercise 4.1.3²: Get the list of all switches which connected to the controller using the following command line:

```
sudo curl -X GET http://0.0.0.0:8080/stats/switches
```

The output is the ZODIAC_ dpid_number, provide the output.

Exercise 4.1.4: Get all flows stats of the switch which specified with Datapath ID in URI using the following command line:

```
sudo curl -X GET http:// 0.0.0.0:8080/stats/flow/ ZODIAC_ dpid_number
```

Provide and explain the output.

Exercise 4.1.5: Get ports stats of the switch which specified with Datapath ID in URI using the following command line:

```
sudo curl -X GET http:// 0.0.0.0:8080/stats/port/ ZODIAC_ dpid_number
```

Provide and explain the output.

Section 4.2: Adding and removing flows

Exercise 4.2.1: Add a flow entry to the switch using the following command line:

```
curl -X POST -d '{
  "dpid": ZODIAC_ dpid_number,
  "cookie": 1,
  "cookie_mask": 1,
  "table_id": 0,
  "idle_timeout": 30,
  "hard_timeout": 30,
  "priority": 11111,
  "flags": 1,
  "match":{
    "in_port":1
  },
  "actions":[
    {
      "type":"OUTPUT",
      "port": 2
    }
  ]
}' http://0.0.0.0:8080/stats/flowentry/add
```

Using ZODIAC-CLI check if there is any additional flow?

² Usage and description of command lines are available at http://ryu.readthedocs.io/en/latest/app/ofctl_rest.html

What is the meaning of the flow?

Exercise 4.2.2: Create a command line for adding a flow entry that allows to link port 2 with port 1. Which is the command line?
Provide the screenshot of the ZODIAC flow table.

Exercise 4.2.3: Delete all flow entries of the switch which specified with Datapath ID in URI using the following command line:
curl -X DELETE http://0.0.0.0:8080/stats/flowentry/clear/ ZODIAC_ dpid_number
Using ZODIAC-CLI what happen with the flows?

Exercise 4.2.4: Add a group entry to the switch using the following command line:

```
curl -X POST -d '{
  "dpid": ZODIAC_ dpid_number,
  "type": "ALL",
  "group_id": 1,
  "buckets": [
    {
      "actions": [
        {
          "type": "OUTPUT",
          "port": 1
        }
      ]
    }
  ]
}' http://0.0.0.0:8080/stats/groupentry/add
```

Using ZODIAC-CLI check the new flow?
Which is the meaning of the flow?

5. Questions

Question 5.1: Explain the advantages of REST API of the Controller.

Question 5.2: Explain and test two commands lines (different from the ones used during the lab) of RYU.APP.OFCTL_REST.

Question 5.3: Provide two real service examples for explaining the usage of reactive flow instantiation.

Question 5.4: Provide two real service examples for explaining the usage of proactive flow instantiation.

Question 5.5: what is the difference between sdn and openflow?

6. References

- [1] RYU Controller. Available at: <https://osrg.github.io/ryu/> [Last access 15/08/2016]
- [2] Zodiac FX, Northbound Networks Pty. Ltd., 2016. [Online]. Available: <http://northboundnetworks.com/collections/zodiac-fx/products/zodiac-fx>. [Accessed: 16-May-2016].